Security of Smart Environments - Notes

IMPORTANT WARNING

- These notes have been made starting from my notes taken during lessons and the power points published by the professor, then rearranged through the help of Gemini to produce a narrative flow.
- All the content belongs to the Professor of the course.
- While I try to ensure everything is inside these notes, please don't use them as a "source of truth", and study from official sources as well.

CHAPTER 1: IoT

Introduction: The Physical Turn in Computer Security

A **smart environment** is a physical space rather than a purely digital domain. It is a home, a hospital, a factory, an office building, a city, or a vehicle that has been instrumented with digital devices capable of bridging the physical and the digital worlds. What distinguishes a smart environment from an ordinary computer network is that the computation is embedded in, and acts upon, the physical space in which people actually live and move.

For this bridge between the physical and digital to exist, two categories of components are indispensable: **sensors** and **actuators**. Sensors collect information from the physical world and translate it into the digital space; actuators perform the reverse operation, transforming digital commands into physical actions. Working together, they allow a smart environment to perceive what is happening around it, reason about that information, take decisions, and then intervene in the physical world.

The behaviour of a smart environment is governed by a small set of defining properties. It exhibits **context awareness**, meaning it understands the situation in which it is operating. It supports **automation**, carrying out actions without continuous human intervention. It provides **adaptation**, adjusting its behaviour to changing conditions. And it permits **remote access**, so that the space can be observed and controlled from outside its physical boundaries. A frequently cited early definition captures the spirit of the concept: a smart environment is a small

world in which many kinds of smart devices work continuously to make the lives of its inhabitants more comfortable (Cook and Das, 2004).

Why Security Failures Become Physical and Personal

The single most important conceptual shift when moving from conventional IT security to smart-environment security is that **the users live inside the system**. This changes the nature of the harm that a security breach can cause.

Consider a compromise of a conventional online service such as a social network. If it is breached, people are upset: they may lose access to their content, or their private information may be exposed. The damage is real but remains within the informational domain. In a smart environment the situation is qualitatively different. Compromising it does not merely threaten the *security* of information; it threatens the *safety* of people. If a nuclear power plant's control system is compromised, the consequences propagate into the physical world, causing damage to cities and to the citizens living in them. The lesson is compact and central to the entire subject: in a smart environment, **security failures become physical, and privacy failures become personal**.

The Ubiquity of Smart Environments

Smart environments are no longer exotic; they are everywhere, and each category carries its own devices and its own risk profile.

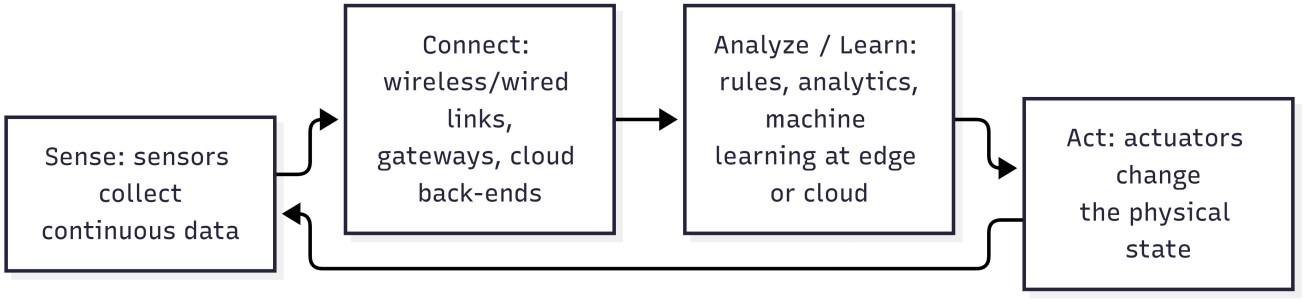
The **smart home** contains voice assistants, cameras, locks, thermostats, and connected appliances. **Smart buildings** manage heating, ventilation and air-conditioning (HVAC), lighting, access control, and elevators, typically through building-management systems and protocols such as BACnet and KNX. At the urban scale, **smart cities and mobility** encompass traffic lights, public transport systems, and connected and autonomous vehicles. The **smart grid and utilities** category includes smart meters, electrical substations, and water-treatment plants. **Smart health** spans wearables, implantable medical devices, and fully connected hospitals. **Smart industry**, the domain of Industry 4.0, is built on programmable logic controllers, industrial robots, and digital twins. Finally, **immersive and mobile spaces** introduce extended-reality (XR) headsets and drones. The breadth of this list is itself a security observation: the attack surface of modern society is now co-extensive with its physical infrastructure.

The Common Operational Pattern: Sense, Connect, Analyze, Act

Despite their diversity, virtually all smart environments operate through the same four-stage cycle. Understanding this loop is essential because each stage is also a distinct opportunity for attack and for defence.

In the **sense** stage, sensors collect continuous data about people and the physical world. This collection is often continuous, though in some designs sensors are triggered only by specific conditions. In the **connect** stage, the heterogeneous devices are linked together through a mixture of wireless and wired connections, passing through gateways and up to cloud back-ends; connection is what makes the environment's information available beyond the physical space itself. In the **analyze** stage, the collected data is interpreted using rules, analytics, and increasingly machine learning, performed either at the edge or in the cloud. In the **act** stage, actuators change the physical state of the world: opening a window, heating a room, moving a mechanism, or applying a brake.

The following diagram expresses this cyclical relationship.



Three things are ultimately at stake in this loop, and together they explain why the security problem is uniquely hard. First, **the data is personal and continuous**: everything sensed can be linked to confidential information about individuals. Second, **the actuation is physical**, which means the concern is safety and not only security. Third, **the scale is enormous**: billions of devices that are cheap, long-lived, and rarely updated. A concrete illustration of the sense-and-act loop is the artificial-pancreas system for diabetes management, in which a continuous glucose sensor feeds readings into a control algorithm that in turn commands an insulin pump; here the intelligent decision is being made about a human body in real time, and an error is not an inconvenience but a medical emergency.

Smart Environments Are Built on the Internet of Things

The building blocks of every smart environment are **IoT devices**, which are characteristically constrained, heterogeneous, and networked. This foundational fact has a direct security consequence: the smart environment **inherits the weaknesses** of its component devices. Default credentials that are never changed, the absence of update mechanisms, and unnecessarily exposed network services all flow upward from the individual device into the environment as a whole.

Worse, the environment does not merely inherit device weaknesses; it **adds entirely new attack surfaces** on top of them. Gateways, cloud back-ends, mobile applications, artificial-intelligence pipelines, and even other people's devices all become potential points of entry. This is why the study of smart-environment security begins from the IoT device itself, understanding what a "thing" is, how it is built and connected, and why it is so difficult to secure, before climbing back up to the architectures, the threat landscape, and the case studies of whole environments.

Defining the Internet of Things

Several complementary definitions of the IoT exist, and each highlights a different facet.

The first, from the European Union Agency for Network and Information Security (ENISA), describes the IoT as a *cyber-physical ecosystem of interconnected sensors and actuators which enable intelligent decision making.* This definition is valuable for foregrounding the cyber-physical nature and the decision-making purpose, though it is notable that it does not itself address the security dimension at all.

A second, more expansive definition comes from the Telecommunication Standardization Sector of the International Telecommunication Union (ITU-T). It characterises the IoT as a *global infrastructure for the information society, enabling advanced services by interconnecting physical and virtual things based on existing and evolving interoperable information and communication technologies.* This definition adds two important clarifications. It notes that, through the exploitation of identification, data-capture, processing, and communication capabilities, the IoT makes full use of things to offer services to all kinds of applications while ensuring that security and privacy requirements are fulfilled. It further observes that, from a broader perspective, the IoT can be understood as a vision with both technological and societal implications.

The Sensor

A **sensor** is an element that allows an IoT system to monitor the environment and the context in which it operates. Sensors measure defined physical, chemical, or biological indicators, and at the digital level they also collect information about the network and the applications running on it. The range of possible measurements is vast: temperature, pressure, body position, airflow, glucose concentration, blood pressure, electrical activity of the heart or muscles, and many others. In a connected-health platform, for instance, a single hub may aggregate readings from a temperature sensor, an electrocardiogram sensor, an airflow sensor, a galvanic-skin-response sensor, a pulse oximeter, a glucometer, a spirometer, and a blood-pressure sensor simultaneously, painting a continuous physiological portrait of a person.

The Actuator

An **actuator** is the entity responsible for moving or controlling a system or mechanism. It operates in the reverse direction of a sensor: whereas a sensor turns a physical quantity into a digital reading, an actuator takes an electrical input and turns it into physical action. Motors, valves, locks, and pumps are all actuators. This complementary relationship between sensing and actuating is what closes the loop between perception and intervention.

The "Thing"

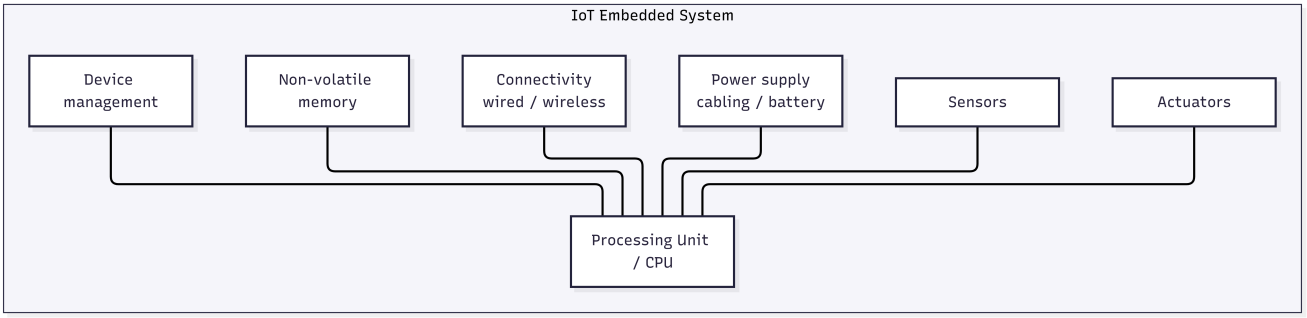
The ITU-T also defines the **thing** at the heart of the IoT. With regard to the Internet of Things, a thing is an object of the physical world (a physical thing) or of the information world (a virtual thing) that is capable of being identified and integrated into communication networks. In practice, a "thing" can be almost anything. It may be a fully capable device such as a server or a smartphone, which is not resource-constrained. More often, however, the thing of interest in security is an **embedded system**.

The Embedded Device

An embedded device is best understood as a small computer dedicated to a specific function. It possesses a central processing unit (CPU) and non-volatile memory in which its program and data persist. It requires connectivity, which may be either wired or wireless, in order to communicate. And it requires a power supply, which may be an energy plug or a battery. Because embedded devices must communicate, and because operators often wish to control the smart environment from outside the physical space, **gateway devices** are introduced. Gateways

forward information to the cloud, from which the state of the smart environment can be accessed remotely.

The internal structure of an IoT embedded system can be visualised as a processing unit at the centre, surrounded by the supporting elements it needs to function.



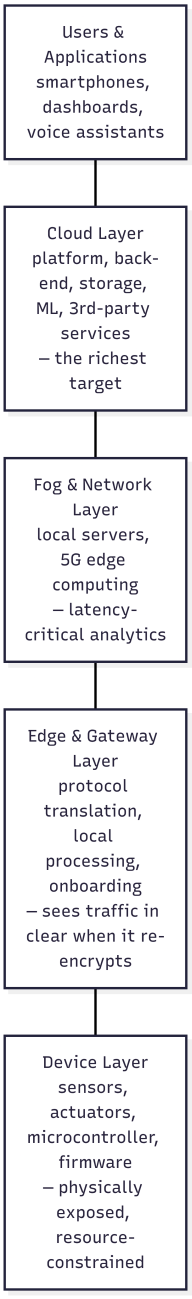
The IoT Ecosystem and Its Layered Architecture

An individual device rarely acts alone. It participates in a broader **IoT ecosystem** in which devices at the bottom, such as connected cars, smartwatches, and washing machines, communicate through gateways, which in turn connect upward to a cloud platform hosting the back-end and services. Communications flow through this hierarchy in both directions, and end users reach the system through smartphones and laptops.

This ecosystem is conventionally organised into a **layered architecture**, and it is essential to grasp that every boundary between layers is simultaneously a **point of attack** and a **place for defence**, where authentication, encryption, and monitoring can be applied.

At the bottom sits the **device layer**, comprising the sensors, actuators, microcontrollers, and firmware. This layer is physically exposed and resource-constrained, which is precisely what makes it vulnerable. Above it is the **edge and gateway layer**, responsible for protocol translation, local processing, and device onboarding. This layer occupies a particularly sensitive position: because it re-encrypts traffic as it passes between the device world and the wider network, it necessarily sees that traffic in clear text at the moment of translation. Next is the **fog and network layer**, consisting of local servers and 5G edge-computing resources that handle latency-critical analytics close to where data is generated. Above that is the **cloud layer**, which hosts the platform, the back-end, storage, machine learning, and third-party services; because it concentrates so much data and capability, it is the richest target of all. Finally, the **users and applications**

layer contains the smartphones, web dashboards, and voice assistants through which people manage the environment.



Because the layers are heterogeneous, so too are the devices within them. Different devices operate under different constraints of energy and computational power, which is the underlying reason the IoT relies on such a wide variety of protocols and standards rather than a single unifying one.

The Scale of the Problem

The number of connected IoT devices has already **surpassed the number of human beings on the planet**, and it continues to climb across every industry, including personal and consumer products, healthcare, automotive, and manufacturing. The growth trajectory is steep: from roughly 7.74 billion devices in

2019, the count rises year by year toward a projected 25.44 billion by 2030 in one widely reproduced estimate.

It is instructive, however, to compare forecasts with reality, because the history of IoT prediction is a history of over-optimism. In 2011, Cisco forecast 50 billion connected devices by 2020; the actual figure turned out to be only about a quarter of that. More recent and more sober estimates from IoT Analytics' *State of IoT 2025* report place the count at 18.5 billion connected IoT devices at the end of 2024, an increase of about twelve percent, with a forecast of 21.1 billion for the end of 2025, an increase of about fourteen percent. Looking further ahead, roughly 39 billion devices are expected by 2030, rising above 50 billion by 2035. Whatever the precise number, the security implication is the same and it is sobering: **each of these devices is a potential entry point, and most of them will never be patched.**

The Protocol and Asset Landscape

The heterogeneity of the IoT manifests concretely in an exceptionally crowded **protocol landscape**. Spanning from the physical link up to business applications, the ecosystem draws on connectivity standards such as Ethernet, RS-232, RS-485, Modbus, USB, and SPI at the lowest levels; short- and medium-range wireless technologies including Bluetooth and Bluetooth Low Energy, Zigbee, Z-Wave, 6LoWPAN, NFC, RFID, and Wi-Fi; wide-area and cellular technologies from 2G through LTE-Advanced and specialised low-power options; and application- and messaging-layer protocols such as MQTT, CoAP, AMQP, XMPP, HTTP, DDS, and REST APIs. These are organised geographically into personal-area networks (PAN, 802.15), local-area networks (LAN, 802.11), metropolitan-area networks (MAN, 802.16), and wide-area networks (WAN, 802.20). Above the communication stack sit data-aggregation, processing, storage, analytics, and business-process functions. The essential takeaway is that this sprawling diversity multiplies the number of implementations, and therefore the number of possible flaws.

The **assets** of an IoT system are correspondingly numerous, and each is a potential target. They include the IoT devices themselves along with their hardware, sensors, actuators, and embedded systems; the infrastructure of routers, gateways, firewalls, and power supply; the security components such as intrusion-detection and intrusion-prevention systems and authentication systems; the platform and back-end comprising cloud infrastructure and web-based services; the decision-making machinery of data mining, big data, algorithms, and machine learning; the information itself in all its states, whether at rest in storage and

databases, in transit across the network, or in use during processing; the communications and their protocols; and the networks spanning LAN, WAN, VPN, and PAN. Any one of these can become the object of an attack.

The State of IoT Security

There is a well-worn joke in the security community that captures the field's central anxiety: someone asks whether the "S" in "IoT" stands for security, and the punch line is that there is no "S" in IoT at all, which is exactly the point. Security was, for a long time, simply absent from the design of connected things.

The history of the security problem is rooted in how these systems evolved. The **initial design** of many industrial and embedded systems assumed a private communication network. Over time these systems were migrated onto IP networks, and later still exposed to the open Internet, a context for which they had never been engineered. The earliest security measures were rudimentary, dealing with authentication through embedded passwords, and it was therefore still simple to find and exploit security faults. Crucially, systems **started with basic security** and only later, once flaws were discovered, had more complex security requirements bolted on. Because of this reactive posture, low-security devices from the early design era are **still deployed today**, and they continue to be used in compatible fall-back modes, meaning a modern system will often downgrade to an insecure legacy protocol simply to remain interoperable. Compounding all of this, **firmware updates are hard or nearly impossible** once devices have been installed, so vulnerabilities discovered after deployment tend to persist for the entire operational life of the device.

The Three Ages of IoT Threats

The evolution of the IoT threat landscape can be divided into three successive ages, each defined by the prevailing attitude toward security and the technologies used to provide it.

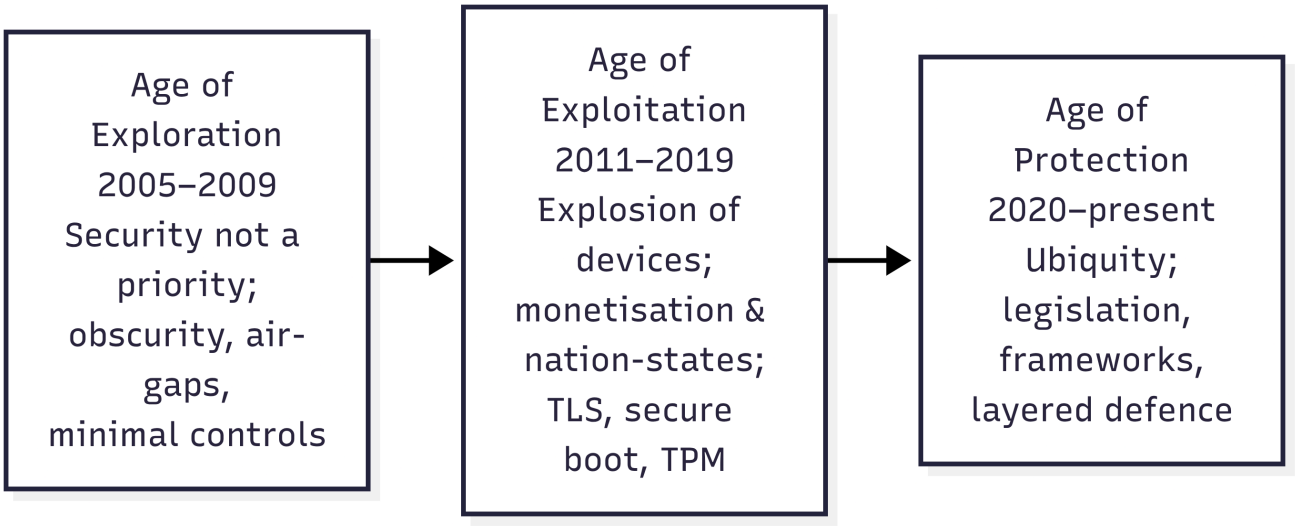
The first is the **Age of Exploration**, spanning roughly 2005 to 2009. In this period, security was not a priority for early IoT and embedded devices. Most cyberattacks were limited to malware and viruses impacting Windows-based embedded control systems. Rather than actively mounting a defence, organisations assumed that no one would bother to attack devices running in isolated networks. The security methods of the era reflected this complacency. Chief among them was **security by obscurity**, the belief that keeping the operating system, application,

or protocol secret would keep it safe; this approach never works, because obscurity is not a substitute for genuine protection. Where security existed at all it was minimal and often easily bypassed. Secure protocols such as SSH or SSL were used in only a handful of systems, and even then usually with no other security controls alongside them. Many networks were **air-gapped**, physically isolated from other networks, and this isolation was itself treated as the primary defence.

The second is the **Age of Exploitation**, spanning roughly 2011 to 2019. During this period the number of connected devices exploded and cloud connectivity became commonplace. Criminals improved their ability to **monetise** attacks on IoT devices through crypto-mining, ad-click fraud, and spam-email campaigns. At the same time, nation-state actors began to use IoT devices for politically motivated attacks. Many new security technologies were being adopted in this age, but their use was inconsistent, incomplete, and sometimes flawed, so a great many devices remained vulnerable in practice. The technologies introduced in this period included security protocols such as TLS and SSH, **secure boot** to verify the integrity of software at start-up, **Trusted Platform Modules (TPM) or Secure Elements** for secure key storage, hardened operating systems, and embedded firewalls.

The third is the **Age of Protection**, beginning in 2020 and continuing to the present. Connected devices are now ubiquitous in every area of life. In response to the growing number and severity of attacks, governments and industrial groups began to enact legislation requiring higher levels of security for IoT devices. Because attackers will always continue to find "soft targets" among both legacy devices and newly deployed devices that lack strong protection, companies worldwide are now building robust security controls directly into IoT devices, using security frameworks and unified solutions in which several key technologies work together to provide multiple, layered defences.

This progression is worth internalising as a single arc, because it explains why the same root causes recur across decades of incidents.



Across this timeline sits a long roster of landmark incidents that mark the maturation of the threat: from the Stuxnet virus and early attacks on smart meters and traffic-light systems, through the compromise of medical devices such as insulin pumps and heart monitors, connected vehicles such as the Jeep Cherokee and the Tesla Model S, and industrial targets such as steel mills and power grids, to the great botnets, Mirai, Hajime, Reaper, Persirai, Amnesia, and their successors, and on to attacks on consumer devices such as the Amazon Ring and the Philips Hue lightbulb. The pattern that emerges is that novelty of the target changes over time, but the underlying weaknesses do not.

Recent Incidents: The Same Root Causes

The persistence of these root causes is starkly visible in the incidents of the current decade. In **2021, the Verkada breach** exposed roughly 150,000 security cameras installed in hospitals, prisons, and factories, all through a single leaked super-admin credential. In **2023, the CyberAv3ngers** group hijacked Unitronics programmable logic controllers in United States water utilities by exploiting default passwords. In **2024, a flaw in Kia's dealer web portal** allowed remote control of millions of cars starting from nothing more than a licence plate. In **2025, BadBox 2.0** saw roughly 10 million Android TV boxes shipped already pre-infected from the factory. Also spanning **2025 and into 2026, the Aisuru and Kimwolf botnets**, both descendants of Mirai, set distributed-denial-of-service records above 30 terabits per second before being disrupted by the United States Department of Justice in March 2026. The common thread through every one of these episodes is identical to that of the previous decade: weak or default **credentials**, unnecessary **exposure** of services, absent **updates**, and vulnerable **ecosystem interfaces**.

Case Study I: Stuxnet

Overview and Strategic Intent

Stuxnet is the pivotal case study in smart-environment security because it was the moment a piece of software crossed the line from the digital into the destructively physical. Its **goal was to sabotage the Iranian nuclear program**, and it succeeded, causing genuine physical damage to the enrichment infrastructure and slowing the nuclear program down by an estimated four years.

The *how* of Stuxnet is what makes it a masterclass in cyber-physical attack. It **interfered with the Industrial Control Systems (ICS)** that were managed by Programmable Logic Controllers, specifically the Siemens Step7 platform. Having gained control, it made the enrichment **centrifuges operate at the wrong speeds**, stressing them toward failure. Most insidiously, while it was doing this, it **presented acceptable, normal-looking values to the monitoring software**, so that the human operators watching their screens saw nothing amiss.

A subtle but important point about the attackers' intent emerges from the impact. The attackers were in a position where they could have simply broken the plant outright; instead, they **preferred to slow the program down**, quietly and over an extended period, which was strategically far more valuable than an obvious act of destruction that would have been immediately detected and repaired. For this reason Stuxnet is widely regarded as the **opening act of cyber warfare**, the first time a state-level cyber weapon produced strategic physical effects.

The Anatomy of the Operation

The overall operation can be pictured as a set of converging elements. The suspected creators, associated with the United States and Israel, developed the weapon. It employed **stolen digital certificates** to sign its components, which helped it avoid suspicion by making its files appear legitimately signed. The malware reached its ultimate destination, the **Natanz enrichment facility**, not over the Internet directly but by being carried across the air gap: plant personnel, themselves manipulated as unwitting carriers, introduced it via **USB removable drives** into the control computers, which in turn were connected to the PLCs governing the centrifuges. Command-and-control communication, where possible, was conducted through innocuous football-themed (futbol-themed) websites to blend in with ordinary traffic.

Statistics and the Signature of a Targeted Weapon

The statistical profile of Stuxnet betrays its targeted nature. The first known sample dates to **2007**, though the malware was not discovered until **2010**, and it was rendered inactive from 2012 onward. It was classified as a **worm**, its targeted platform was **industrial SCADA systems**, and its infections numbered somewhere between ten thousand and three hundred thousand hosts. The overwhelming concentration of those infected hosts was in **Iran**, dwarfing every other country, and this geographic concentration is itself the fingerprint that reveals Iran to have been the intended initial target.

Stuxnet is often described as a **"fire and forget" weapon**: once released, it operated autonomously without needing further direction. Although it was fundamentally a targeted threat, the propagation techniques it used to reach an air-gapped facility inevitably let it **spread beyond its intended target**, producing what is aptly termed **"collateral damage"** as it appeared on machines around the world that had nothing to do with Natanz. The scale of this spread relative to its origin is striking: by February 2011, some 12,000 infections could be traced back to only **10 initial infections in 5 organisations in Iran** between 2009 and 2010, illustrating how a tiny, precisely aimed seed could blossom into a global presence.

The Installation Routine: Export 15

Stuxnet's main library exposed its functionality through numbered exports, two of which govern its life on a newly reached machine. The first, **Export 15**, is the installation routine, and it performs a careful sequence of environmental checks before committing to infection. This defensiveness is deliberate: a state-level weapon must avoid detection and avoid wasting itself on unsuitable or suspicious hosts.

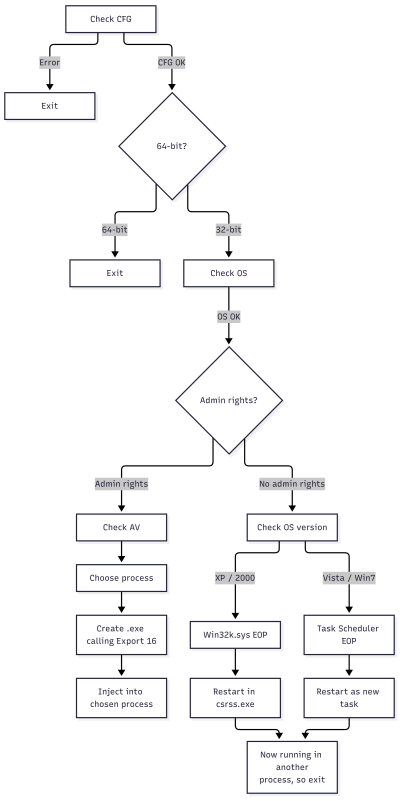
The routine begins by **checking the machine's configuration (Check CFG)**. It inspects the victim's PC to see whether anything about the machine looks strange or unexpected, and if it detects an anomaly it exits immediately rather than risk exposure. If the configuration is acceptable, it next **checks whether the machine is 64-bit**. If the machine is 64-bit, Stuxnet stops. The reason is a precise piece of targeting intelligence: while by 2010 most of the world had migrated to 64-bit systems, the machines in the Iranian facility were still running 32-bit Windows. By refusing to run on 64-bit systems, Stuxnet effectively

restricted itself to infecting only the 32-bit Windows PCs that matched its intended environment.

Having confirmed a 32-bit machine, the routine then **checks the operating system**, and subsequently **checks whether the current user has administrative rights**. The two branches that follow depend on the answer to this privilege question.

If the user **does have administrative rights**, Stuxnet proceeds to **check the antivirus (Check AV)** installed on the machine. It uses knowledge of the specific antivirus product to decide how to inject itself safely, then **chooses a process** to inhabit, **creates an executable that calls Export 16**, and **injects itself into the chosen process**, thereby beginning the true infection.

If the user **does not have administrative rights**, Stuxnet must escalate its privileges, and it does so by exploiting one of two elevation-of-privilege vulnerabilities selected according to the operating system version. On **Windows XP or Windows 2000**, it exploits a vulnerability in **Win32k.sys**, after which the threat restarts within the `csrss.exe` process. On **Windows Vista or Windows 7**, it exploits a vulnerability in the **Task Scheduler**, after which it restarts as a new task. In either case, the elevation gives it the administrative rights it lacked, the threat is now running in another process, and the original instance exits.



The Infection Routine: Export 16

The second export, **Export 16**, carries out the infection proper, and like the installation routine it opens with a set of guard conditions before doing anything destructive.

It first **checks the configuration** again, exiting on error. It then inspects a specific **registry key**: if the value `NTVDM Trace` is set to the value **19790509**, the malware exits. This is one of the most famous details of the entire case. The value is a date, 9 May 1979, and its presence functions both as an infection marker and, in the eyes of analysts, as a possible signature; it has been read as an allusion that supports attribution of the attack's origins. Next, the routine performs a **date check**, verifying that the current date is before **24 June 2012**; this establishes a hard **deadline**, a kill date after which the malware would cease to operate, and if the date is past that deadline it exits.

Assuming these checks pass, the routine **checks the operating system** and, depending on the version, sets access-control structures on its files, using a **DACL** on Windows XP or earlier and a **SACL** on Windows Vista or higher. It then **creates a global mutex** to ensure only one instance runs, and **creates its `.pnf` and `.cfg` files** on disk. It performs a further **date check** against the same 24 June 2012 deadline, then **decrypts and loads itself from disk, calling Export 6 to retrieve its version number**.

At this point a version-management step occurs, which is central to how Stuxnet kept itself current across an isolated network. It **compares the running version number against the version present on disk**; if the version is acceptable, it **decrypts resources 201 and 242 and writes them to disk**, then **sets the file times** to blend in, and **creates the rootkit-service registry keys** whose purpose is to **hide the malicious files**. It then **creates global mutexes** and proceeds to three parallel infection activities: it **infects removable drives** and **injects into services, calling Export 32**; and, along a second path, it **injects into Step7 and calls Export 32** in order to **infect Step7 projects**. Two of the rootkit files it deploys in this process are named **Mrxcls.sys** and **Mrxnet.sys**. When these steps are complete, that branch exits.

The essential conceptual takeaway from Export 16 is that Stuxnet was not merely a piece of malware that spread; it was a self-managing, self-hiding, version-aware system with a built-in expiry date, engineered to remain quietly resident and current inside a network it could not reach from the outside.

Stuxnet Propagation: Three Avenues of Spread

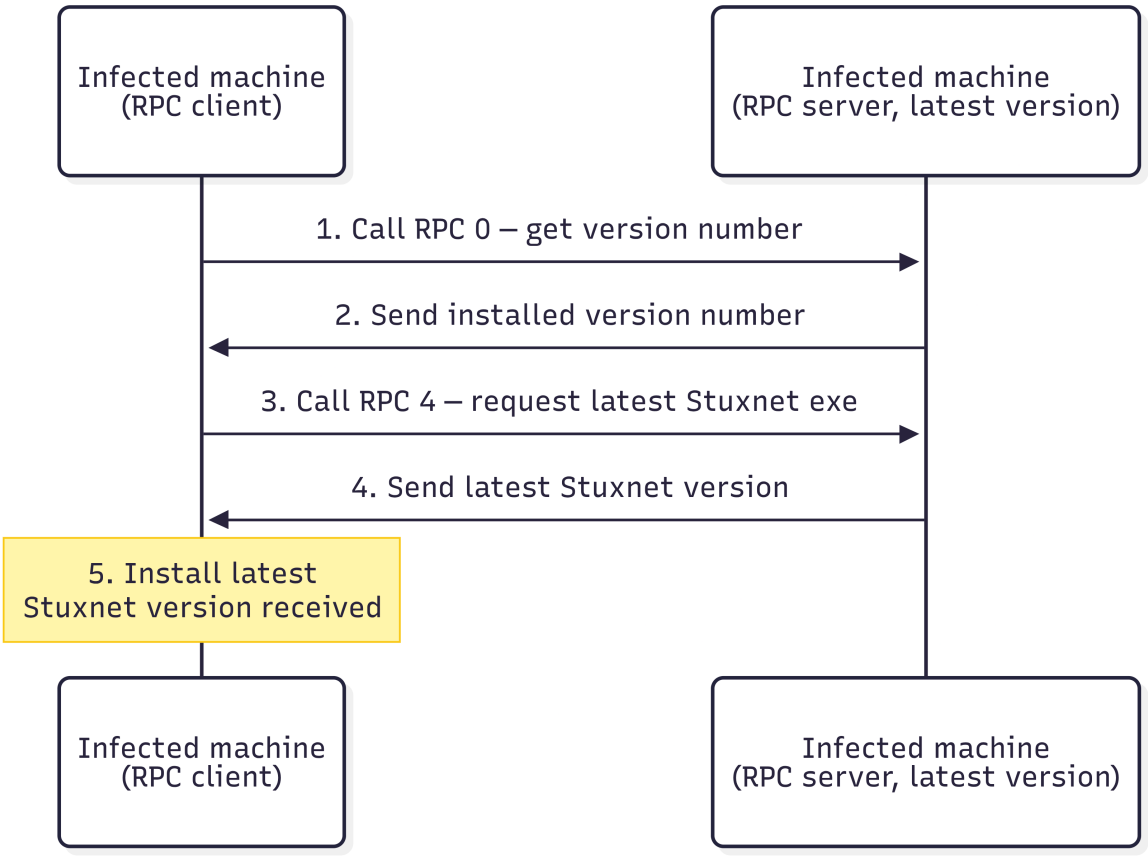
Reaching an air-gapped facility required Stuxnet to be exceptionally versatile in how it moved from machine to machine. It employed three distinct propagation strategies, each suited to a different part of the journey from the outside world to the isolated PLCs at Natanz: **network propagation**, **removable-drive propagation**, and **Step7 project-file infection**.

Peer-to-Peer Network Propagation and Self-Updating

The most sophisticated aspect of Stuxnet's network behaviour was its ability to **update itself across a network of compromised machines**, without any central server, using a peer-to-peer mechanism. This was indispensable for an operation targeting an isolated network: once a newer version of the malware managed to reach even one machine, that version could propagate itself to all the older instances already present, keeping the entire infected population synchronised on the latest code.

The mechanism worked by installing both an **RPC server and an RPC client** on each infected machine. The server was started and listened for incoming connections, while the client searched the network for other Stuxnet servers and initiated communication with them. When two infected machines established contact, they compared versions, and any **older instance was updated to the newer one**. As always with Stuxnet, the update was not executed naively; it was **injected into another process** to remain concealed.

The update exchange followed a defined request-response sequence between an infected machine acting as the RPC client and another acting as the RPC server:



The client first asks the server for its version number; the server replies with its installed version; if the server's version is newer, the client requests the latest executable; the server transmits it; and the client installs the received version. Through this quiet, decentralised gossip, updates diffused throughout the network of compromised machines.

Propagation Through Network Shares

A second network avenue exploited Windows file sharing. Stuxnet **enumerated all user accounts** of both the local computer and the domain. It then attempted to reach **all available network resources**, using either the user's own credential token or **Windows Management Instrumentation (WMI)**, in order to copy itself onto the remote share and execute there. By walking the list of accessible shares with legitimate credentials, it could move laterally through an organisation's internal network from any foothold it had already established.

The Microsoft Vulnerabilities Exploited

To copy itself onto remote machines that were not otherwise reachable, Stuxnet weaponised two specific Microsoft vulnerabilities.

The first was **MS10-061**, a zero-day vulnerability in the **Print Spooler** service. Exploiting it, Stuxnet copied itself to remote computers as

`%System%\winsta.exe` by way of the Printer Spooler, and then executed itself there. Because this was a zero-day at the time, no patch existed and no defender was prepared for it.

The second was **MS08-067**, a vulnerability in the **Windows Server Service**. Here Stuxnet connected over the **Server Message Block (SMB)** protocol and sent a **malformed path string** that permitted arbitrary code execution, allowing it to copy itself to unpatched remote computers. Between the peer-to-peer updating, the network-share enumeration, and these two remote-execution exploits, Stuxnet possessed a full toolkit for silent lateral movement through a Windows network.

Removable-Drive Infection

Because the target network was air-gapped, the truly decisive propagation vector was the **removable drive**, since a USB stick carried by a human being was the only realistic way to cross the physical isolation into the plant.

The LNK Vulnerability (CVE-2010-2568)

Three of Stuxnet's exports, numbered **1, 19, and 32**, were responsible for adding or removing the infection on a removable drive. The malware waited for a removable drive to be inserted, verified that it contained a logical volume and was genuinely a removable drive, and if the drive was deemed a suitable candidate, infected it. A notable piece of operational discipline governed this behaviour: **the infection was removed from a drive after it had already infected three PCs**, limiting uncontrolled spread and reducing the chance of discovery.

When infecting a drive, Stuxnet created a specific set of files. It placed a large temporary file, `~WTR4132.tmp` of roughly 500 kilobytes, which **contained Stuxnet's main DLL in its stub section**, alongside a smaller loader file, `~WTR4141.tmp` of roughly 25 kilobytes, whose job was to **load `~WTR4132.tmp`**. It then created a series of shortcut files with escalating names: `Copy of Shortcut to.lnk`, `Copy of Copy of Shortcut to.lnk`, `Copy of Copy of Copy of Shortcut to.lnk`, and `Copy of Copy of Copy of Copy of Shortcut to.lnk`. There were several of these because each one contained an exploit crafted for a different version of Windows, ensuring that whichever operating system the target machine ran, one of the shortcuts would trigger. These shortcuts exploited the **LNK vulnerability, CVE-2010-2568**, which caused the malicious code to execute merely when the drive's contents were displayed, without the user having to knowingly run anything.

The AutoRun.Inf Vector

An alternative activation method relied on the `autorun.inf` mechanism. A key observation here is that **`autorun.inf` parsing in Windows is quite forgiving**: any characters that are not understood as legitimate autorun directives are simply ignored. Stuxnet exploited this leniency by constructing a file that was simultaneously a valid executable and a valid autorun configuration; the executable content of the malware could be padded into the same file that Windows parsed as autorun instructions, because Windows discarded the parts it did not recognise. One visible symptom of this technique was that the right-click context menu on the infected removable disk showed **two "Open" commands** instead of one, both of which led to executing the autorun payload. Through either the LNK exploit or the autorun mechanism, simply plugging in and viewing the drive was enough to trigger infection.

Infecting WinCC Computers

Once inside the plant's network, Stuxnet sought out a specific piece of industrial software: **SIMATIC WinCC**. WinCC is a supervisory control and data acquisition (SCADA) system with a human-machine interface (HMI), produced by Siemens. It is used to monitor and control the physical processes involved in industry and infrastructure on a large scale and over long distances, and it holds the historical distinction of being among the **first known SCADA systems to be specifically targeted by malware**.

When Stuxnet detected a WinCC installation, it carried out a precise sequence of database attacks. It first **connected to the database server using a password hardcoded within the WinCC software itself**, a credential that was fixed and known, and which no operator could realistically change. It then **sent malicious SQL code to the database**, infecting the computer running the WinCC database. It **modified an existing database view**, adding code that would be executed every time that view was subsequently accessed, guaranteeing repeated re-execution of its payload. Finally, it **sent an SQL statement that created a new table and inserted into it a hexadecimal-string representation of the main Stuxnet DLL as an executable file**, identified as resource 210. The hardcoded database password here is a textbook example of exactly the kind of credential weakness that recurs throughout the history of IoT and industrial security.

Step7 Project Infection

The third propagation route targeted the engineering software used to program the controllers, **Step7**. Stuxnet **infected the APIs used to manage a WinCC or Step7 project**, and these infected APIs were the ones typically invoked to **open `.s7p` project files and `.mcp` project components**. In this way, merely opening an engineering project could trigger the infection.

Operationally, Stuxnet took a path to a project, or a compressed archive of a project, as its input. It **listed all the files inside that path**, **validated which of them were candidates for infection**, and then **infected the qualifying files by injecting three things into them**: an encrypted copy of the main Stuxnet DLL, a copy of a Stuxnet data file, and an encoded, updated version of the Stuxnet configuration data block. This meant that infected engineering projects became carriers, propagating the malware to any other engineering workstation on which they were later opened.

The Final Target: The PLC

Every propagation mechanism described so far served a single ultimate purpose: reaching and subverting the **Programmable Logic Controller (PLC)**. A PLC is an industrial digital computer adapted specifically for the control of manufacturing processes. In the Natanz scenario it was the targeted device, and PLCs in general are notable for being **highly fault-tolerant** and for being **programmed through dedicated software**, in this case Step7.

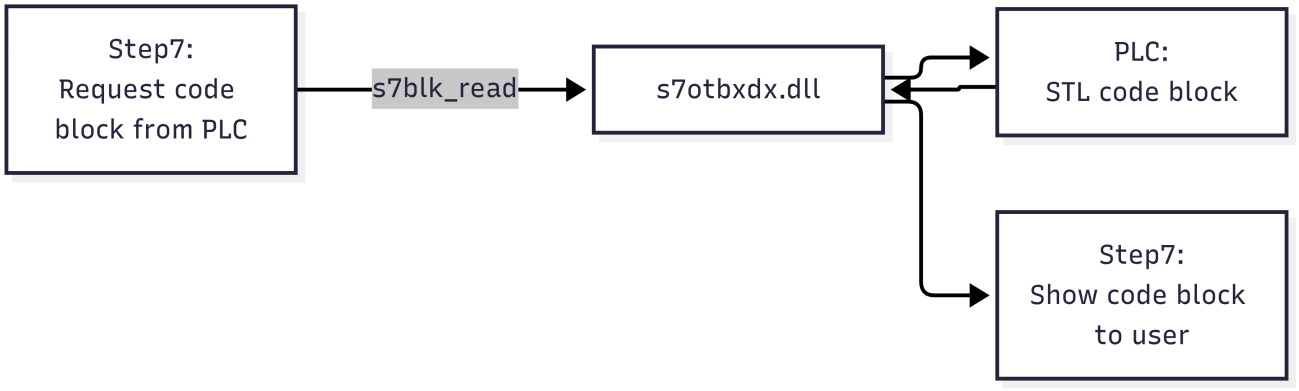
The normal relationship between the engineering workstation and the controller is straightforward. The **Step7 control software** runs on a control PC, and it issues **read and write requests** to the **PLC**, which stores and executes its control logic in the form of **STL code** (Statement List, a low-level PLC programming language).



How Step7 Communicates With the PLC

When a PLC is connected to the workstation by cable, the Step7 software is permitted to perform a range of privileged operations: it can **access the memory content** of the PLC, **reconfigure it**, **upload code into it**, and **debug the code running on it**. All of this communication is mediated by a single, critical library file named **`s7otbxdx.dll`**, which **contains all the necessary communication routines** between Step7 and the PLC. In normal operation,

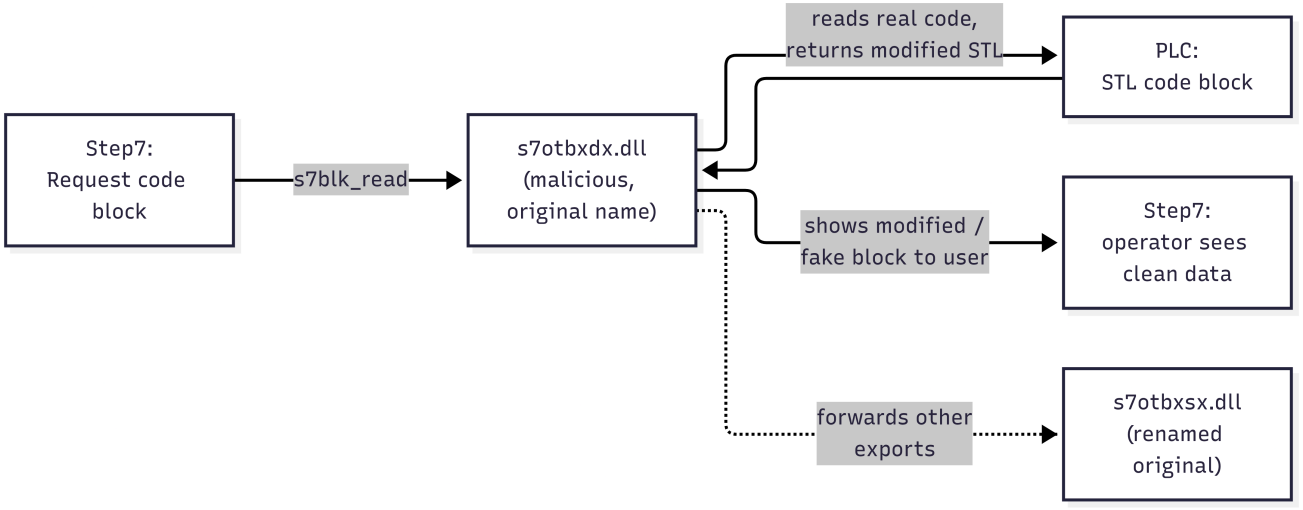
when Step7 requests a code block from the PLC, the request passes through this DLL's `s7blk_read` function to the PLC, the PLC returns its STL code block, and the DLL passes that block back up to Step7 for display to the user.



Modifying the PLC: The Man-in-the-Middle DLL

Stuxnet's method for seizing control of the PLC was an elegant and devastating library-substitution attack. It **renamed the legitimate `s7otbxdx.dll` to `s7otbxsx.dll`** and **replaced the original with its own malicious version** bearing the original name. The malicious DLL was carefully constructed to **contain all the exports of the original**, so that Step7 would continue to function normally and suspect nothing. The **majority of these exports were simply forwarded to the renamed legitimate library** (`s7otbxsx.dll`), preserving genuine behaviour, while **sixteen exports were replaced** with Stuxnet's own implementations.

Those sixteen replaced exports gave Stuxnet **control of the read, write, and enumerate functions**, which is precisely the set of capabilities needed to dominate the controller. As a result, Stuxnet could **write its own code into the PLC**, **change operating parameters**, **hide its malicious code from anyone inspecting the controller**, and, critically, **provide fake information to the operator**. This last capability is what allowed the attack to proceed undetected: when an engineer used Step7 to read back the code or the state of the PLC, the malicious DLL intercepted the request and returned a clean, expected result, concealing the modifications underneath.



This man-in-the-middle position between the engineering software and the physical controller is the heart of the entire Stuxnet operation. Having achieved it, the malware could rewrite the logic governing the centrifuges while showing the operators a reassuring fiction.

The PLC Infection Process

With control of the communication library established, Stuxnet turned to the actual infection of the controller. The threat contained **three distinct infection sequences**, conventionally labelled A, B, and C. **Sequences A and B had similar functionality but were aimed at different targets**, while **Sequence C appeared to be disabled or only partially completed**, suggesting either an abandoned line of attack or a component never fully deployed.

The mechanics of infection were driven by the malicious library, which **started two threads**. The **first thread ran an infection routine, either sequence A or sequence B, every fifteen minutes**, repeatedly attempting to compromise the controller. The **second thread queried the PLC for a specific block that would have been injected by the first thread**, but only if the infection process had already succeeded. This injected block was **customised**, and its presence and content **impacted the way sequences A or B ran on the infected PLC**, allowing the attack to tailor its behaviour to the particular controller it had compromised.

The Infection Sequence in Detail

Each infection sequence unfolded through a series of stages that together transformed a legitimate controller into a covert saboteur.

The sequence began with a **System Data Block check**. The PLC's blocks were **enumerated and parsed**, and the malware **searched for PLCs that used**

specific frequency-converter drives. This targeting detail is crucial: Stuxnet was not interested in any PLC, but only in those controlling the particular type of variable-frequency drives used to spin the enrichment centrifuges. This selectivity is what confined the destructive payload to its intended victims.

Next came the **replacement of communication protocols**. From this point on, **all communications received from or sent to the PLC were processed by Stuxnet before delivery**, cementing the man-in-the-middle position at the level of the controller's own traffic and enabling both the manipulation of commands and the falsification of readings.

Then came the **modification of Organization Blocks**. Organization Blocks are in charge of coordinating the activity of the operational blocks within the PLC's program. Stuxnet arranged for its **malicious code sequence to be executed at the start of a cycle**, hijacking the controller's regular scan cycle, and it took measures to **prevent the blocking of its malicious code execution**, ensuring its payload could not be easily stopped. Finally, the **malicious blocks were written into the PLC**, completing the subversion of the controller's logic.

What Happened: The Physical Sabotage

The purpose of all this machinery was a specific, patient act of physical sabotage against the centrifuges, executed through two attack routines that each followed a similar five-step rhythm of watch, wait, strike, and reset.

In the first routine, the **frequency-converter slaves sent records about their operating frequencies**, and Stuxnet **examined these frames and recorded the relevant fields**, quietly building a picture of normal operation. After **approximately thirteen days**, enough events had been recorded to establish that the system had been **operating correctly between 807 Hz and 1210 Hz**. Only then did the malware strike: the infected PLC **commanded its frequency-converter drives to set the frequency to 1410 Hz**, well above the normal operating range, before shortly **dropping the frequency to 1064 Hz**, after which the **PLC reset its variables** and the cycle began again.

The second routine followed the same logic with different timing and values. The **PLC collected information**, and after **approximately twenty-seven days**, once enough events had been recorded, the infected PLC **commanded its frequency-converter drives to operate at 2 Hz**, a near-halt, before commanding them **back to 1064 Hz**, and again the **PLC reset its variables**.

The physical significance of these frequency manipulations is what turned software into a weapon. Centrifuges are precision devices spinning at enormous speeds; driving them far above their normal frequency and then abruptly far below it subjects them to mechanical stresses that damage or destroy them over time. Crucially, because the changes were interspersed with long periods of normal operation and separated by many days, they resembled sporadic equipment faults rather than a coordinated attack.

What the Engineers Observed

From the perspective of the plant's engineers, the situation was maddeningly opaque, and this opacity was itself the masterstroke of the attack. The engineers were **coming to understand that the turbines were operating over-speed and under-speed**, sensing that something was physically wrong with the centrifuges. Yet **the PLC, under Stuxnet's control, reported that everything was within the limits**, because the malicious library was feeding the monitoring software fabricated, acceptable values. Compounding their helplessness, any **actual manipulation of the legitimate control logic that the engineers attempted had no effect**, since Stuxnet sat between their commands and the hardware, intercepting and neutralising their interventions. The engineers were, in effect, locked out of their own plant while being shown a screen that insisted all was well, the precise embodiment of what it means for a security failure in a smart environment to become a physical failure.

Stuxnet's Legacy: A Modern Echo

The lessons of Stuxnet are thrown into sharp relief by a much more recent and far cruder incident that struck the same class of target. On **25 November 2023**, a booster station of the **Municipal Water Authority of Aliquippa**, in Pennsylvania, was hijacked, and its pumping had to be switched to manual operation. The target was a **Unitronics Vision PLC** whose human-machine interface was **exposed directly on the Internet**, on **TCP port 20256**, protected by nothing more than the **default password 1111**. The attacker was a group called **CyberAv3ngers**, affiliated with the Iranian Revolutionary Guard Corps; their motive was that the equipment was Israeli-made, and they defaced the device's screen. This was part of a broader campaign of **four waves of attacks against Unitronics devices** in the water, energy, food, and healthcare sectors, documented in **CISA advisory AA23-335A**.

The contrast with Stuxnet is the entire lesson. **No exploit was needed at all**: the device simply did exactly what it had been configured to do for anyone who supplied the default password. Where Stuxnet represented the pinnacle of sophistication, requiring stolen certificates, multiple zero-days, and a bespoke man-in-the-middle library, Aliquippa required only a guess. It stands as a demonstration of **OWASP IoT risk number one, weak, guessable, and hard-coded passwords, appearing in critical infrastructure five years after that list was published**. The sophistication of the attacker had collapsed to nearly nothing, and yet the impact on a physical utility was immediate, precisely because the underlying weaknesses of industrial smart environments had not changed.

The Kia Portal Case: When the Weakest Link Is the Ecosystem

A second modern case demonstrates a different but equally important lesson: the most vulnerable part of a smart environment is frequently not the device at all, but the **ecosystem interface** built around it. In **June 2024**, a group of researchers, Curry, Rivera, Rhinehart, and Carroll, discovered that they could **control any Kia vehicle manufactured after 2013 starting from nothing more than its licence plate**.

The attack was a chain of steps, none of which involved compromising the car itself. It exploited the **dealer web portal**. The researchers were able to **create a fake dealer account** on that portal, which granted them access to an **API that mapped a licence plate to the vehicle identification number (VIN), and the VIN to the owner**. Using this, the attacker could simply **add themselves as a second authorised user of the vehicle**. The effects were immediate and severe. In **under thirty seconds**, an attacker could **locate, unlock, start, and honk** the target car, and could also retrieve the **owner's name, phone number, e-mail address, and home address**. Notably, **no active subscription to any connected-car service was required** for this to work. Kia fixed the flaw in **August 2024**, and there is no evidence it was ever exploited maliciously in the wild.

The lesson is the essential complement to Stuxnet's. The car was fine; the portal was not. **The weakest link of a smart environment is often its ecosystem interface, corresponding to OWASP IoT risk number three, insecure ecosystem interfaces, rather than the device itself.** A perfectly secure vehicle can be wholly compromised through a badly designed web application that

surrounds it, because in a smart environment the boundary of the attack surface extends far beyond the physical object into every supporting service, API, and back-end that touches it.

Case Study II: Mirai and the Botnet Threat

The Mirai Malware

Where Stuxnet was a precision instrument of state sabotage, **Mirai** represents the opposite pole of the threat landscape: mass, indiscriminate compromise of cheap consumer devices for the purpose of raw disruptive power. Mirai **turns networked devices running Linux into remotely controlled bots that can be marshalled as part of a botnet operating at large scale**. It **primarily targets online consumer devices such as IP cameras and home routers**, the very sort of inexpensive, rarely-updated equipment that populates smart homes by the billion. Its speed was extraordinary: in its early operation it **infected nearly 65,000 devices in just 20 hours**, a rate of growth that illustrates how quickly a self-propagating worm can weaponise the consumer IoT.

Understanding Bots and Botnets

To understand Mirai it is necessary to be precise about the vocabulary of botnets. A **bot** is a program that secretly takes over another Internet-attached computer and then uses that computer to launch attacks that are difficult to trace back to the bot's true creator; a compromised machine of this kind is also known as a **zombie** or a **drone**. Bots are typically **planted on hundreds or thousands of devices belonging to unsuspecting third parties**, who have no idea their equipment is participating in an attack. A **collection of bots acting in a coordinated manner is a botnet**.

A botnet, to function as such, exhibits **three defining characteristics**: the **bot functionality** itself, meaning the malicious capability installed on each device; a **remote-control facility**, meaning the command-and-control channel through which the operator directs the bots; and a **spreading mechanism** that propagates the bots and thereby constructs and grows the botnet.

The Many Uses of Bots

Once an attacker commands a botnet, the aggregated resources of many devices can be turned to a wide range of malicious ends. The most notorious use is the **distributed denial-of-service (DDoS) attack**, which causes a loss of service to legitimate users by overwhelming a target. Botnets are also used for **spamming**, sending massive volumes of bulk e-mail, and for **sniffing traffic**, in which a packet sniffer running on the compromised device retrieves sensitive information such as usernames and passwords. They perform **keylogging**, capturing the keystrokes of the device's legitimate user, and they are used for **spreading new malware**, since an existing botnet is an ideal delivery network for new bots.

Beyond these, botnets support **installing advertisement add-ons and browser helper objects (BHOs)**, in which the attacker sets up a fake website and negotiates a deal with hosting companies that pay for advertising clicks, then uses the bots to generate fraudulent clicks. They are used for **attacking Internet Relay Chat (IRC) chat networks**, flooding a victim with requests to bring down the IRC network in a manner similar to a DDoS attack. And they are used for **manipulating online polls and games**, exploiting the fact that every bot has a distinct IP address so that each fraudulent vote or action appears to come from a distinct real person.

The Mirai Event Timeline

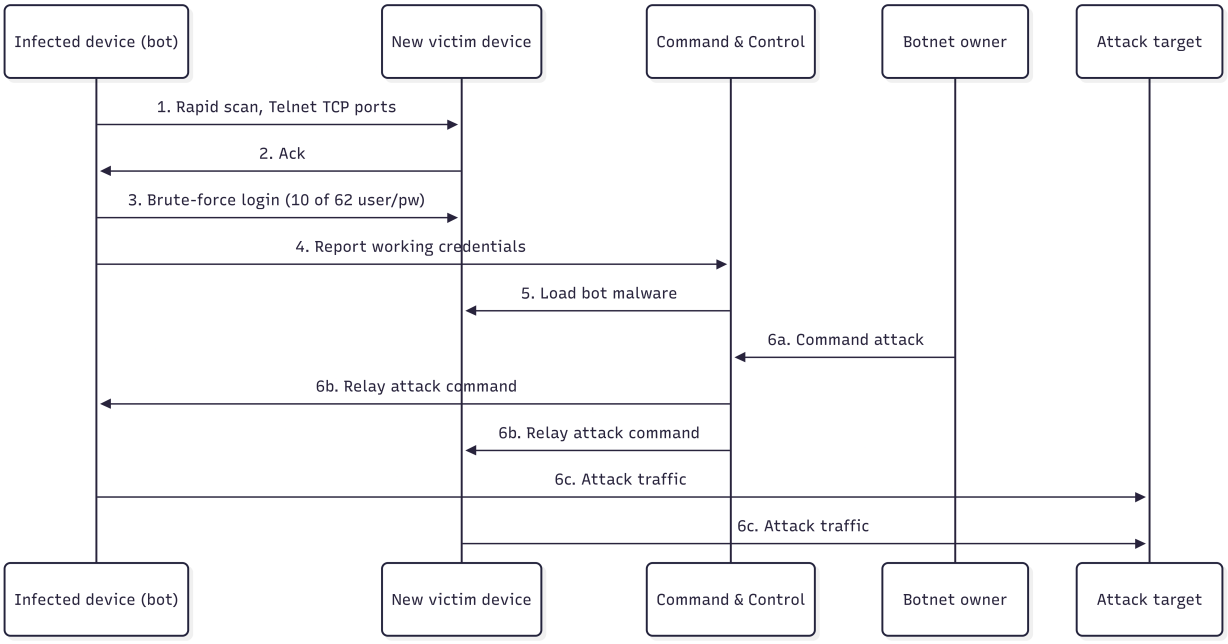
Mirai's public history unfolded rapidly across late 2016 and into 2017. The malware **surfaced on 1 August 2016**. Its first large attacks came against the hosting provider **OVH on 18 September 2016**, followed by a **peak attack on the security journalist Brian Krebs's site (Krebs on Security) on 21 September 2016**. On **30 September 2016 the Mirai source code was released** publicly, a pivotal moment whose consequences are discussed below. The attacks against the DNS provider **Dyn occurred on 21 October 2016**, and an attack against **Liberia's Lonestar network began on 31 October 2016**. A new exploit, the **Deutsche Telekom CWMP exploit, appeared on 26 November 2016**. The **author of Mirai was identified on 18 January 2017**, and a **Deutsche Telekom attacker was arrested on 23 February 2017**.

How Mirai Operates: Scanning, Brute-Forcing, and Recruiting

The operational cycle by which Mirai grows and then attacks can be reconstructed step by step. Imagine an already-infected IP camera. That compromised device begins by performing a **rapid scan of the network, probing Telnet on TCP**

ports, searching for additional devices to infect. When it finds a candidate that responds, it attempts to log in through a **brute-force process**, trying a small dictionary of credentials. A key efficiency choice governs this step: Mirai typically tried only about **ten passwords out of a list of sixty-two possible username-and-password combinations per device**, deliberately limiting the effort so as not to waste time on any single target and instead infect a larger set of devices overall.

When a valid password is found, the compromised device **sends a report to the command-and-control (C2) infrastructure**, communicating the credentials and address of the newly discovered vulnerable device. The **C2 then arranges for the Mirai bot to be loaded onto the new device**, infecting it and enrolling it in the botnet. Finally, when the operator wishes to strike, the **owner of the botnet communicates through the C2 to the assembled devices, commanding them to launch the attack** against a chosen victim.



This self-amplifying loop, in which each newly infected device becomes a scanner recruiting still more devices, is what allowed Mirai to grow to tens of thousands of bots within a single day.

The Dyn Botnet DDoS Attack

The most consequential deployment of Mirai was the **distributed denial-of-service attack of October 2016 against the DNS provider Dyn**. Because Dyn provided domain-name resolution for a large portion of the Internet, taking it down had cascading effects, and this attack was **staged and launched from IoT devices running Mirai**. It became the defining demonstration of how the

compromised consumer IoT could be turned into a weapon capable of disrupting major online services.

Mirai was engineered for **two main purposes**. The first was to **find and infect IoT devices in order to grow the botnet**, the self-propagating behaviour described earlier. The second was to **participate in DDoS attacks based on commands received from the remote command-and-control (C2) infrastructure**. To serve these ends, Mirai operates in **three stages**: it infects the device, it protects itself, and it launches the attack.

Stage 1: Infect the Device

In the first stage, Mirai **scans for IoT devices that are accessible over the Internet**. It **primarily scans for open ports such as 22, 23, and 5747**, among others, though it **can be configured to scan for additional ports**. Port 23 is Telnet and port 22 is SSH, the remote-access services through which the malware seeks entry. Once it connects to a device, it **brute-forces usernames and passwords to log in**, and having gained access it **uses that device to scan networks looking for still more IoT devices**, perpetuating the recruitment cycle.

Stage 2: Protect Itself

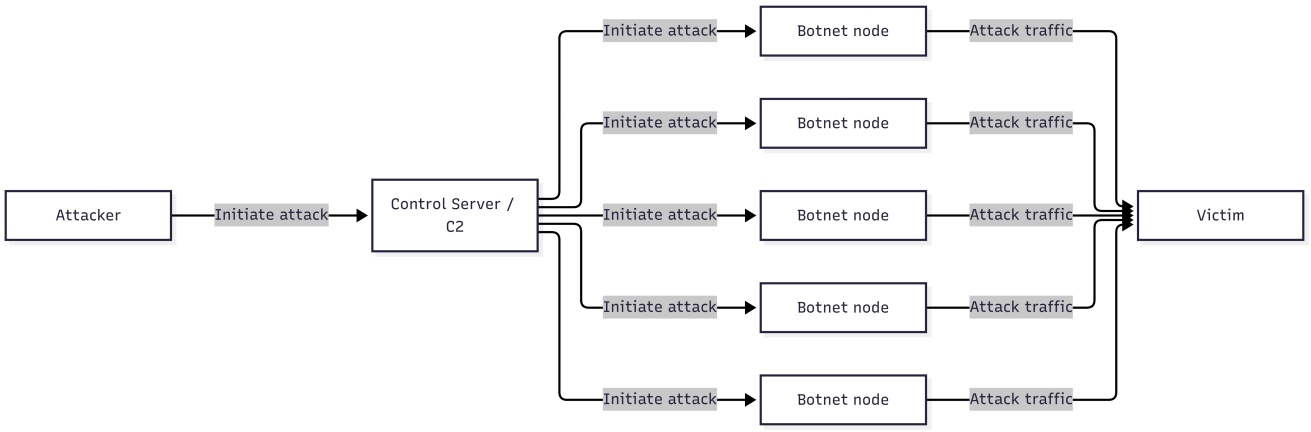
In the second stage, Mirai acts to secure its hold on each newly captured device. It **kills other processes running on the infected device, specifically SSH, Telnet, and HTTP**, in order to **prevent the legitimate owner from gaining remote access to the device while it is infected**. This both denies the owner the ability to detect and remove the malware and prevents rival malware from claiming the same device. An important practical note accompanies this stage: **rebooting the device can remove the malware**, because Mirai resides in volatile memory and does not achieve persistence, **but the device can simply become infected again** almost immediately, since the underlying vulnerability, the weak default credential, remains unchanged after the reboot.

A further notable design detail is that **Mirai contained a hardcoded list of known networks in the United States that it was instructed to avoid attacking**, among them the **United States Postal Service and the Department of Defense**. This deliberate avoidance of certain high-profile government networks is a recurring observation about the malware's behaviour.

Stage 3: Launch the Attack

In the third stage, upon command, the **infected device launches different types of attacks**. These included **HTTP floods and SYN floods**, among other **DDoS-based techniques**. In the Dyn campaign the assault took the form of **twenty-one short-lived attacks of about twenty-five seconds each, followed by two long attacks**, and it targeted **the Dyn infrastructure and the PlayStation infrastructure**.

The overall architecture of such an attack can be visualised as a hierarchy flowing from a single human attacker down through control infrastructure to a mass of geographically dispersed bots, all converging their traffic on one victim.



The key structural insight of this diagram is the separation between the attacker and the attack. The traffic that strikes the victim originates not from the attacker's own machine but from thousands of innocent third-party devices, which is precisely what makes a DDoS attack both powerful, because of its aggregate volume, and difficult to trace, because the true originator is hidden behind the botnet.

The Creators of Mirai

The origin story of Mirai carries its own ironic lesson. The botnet was created by **Paras Jha and Josiah White**, who were **both twenty years old** at the time. Tellingly, they had **co-founded Protraf Solutions, a company that offered mitigation services for DDoS attacks**. In other words, the creators of one of history's most disruptive DDoS tools were simultaneously in the business of selling protection against DDoS attacks, an arrangement that blurred the line between the arsonist and the fire brigade.

Why Mirai Remains Dangerous

Mirai did not end with the arrest of its creators, and understanding why is essential. The malware **remains dangerous because it is mutating**, and above

all because **its source code lives on**: it was published publicly and remains available in repositories online, notably a well-known GitHub mirror. Once the source code was released, any attacker could take it, modify it, and deploy a new variant, which is exactly what happened.

Consequently, Mirai **has given birth to a family of variants**, including **Okiru, Satori, Masuta, PureMasuta, and IoTrooper and Reaper**. Each of these descendants inherited Mirai's fundamental approach while adding new capabilities or targeting new devices. The public release of the source code transformed Mirai from a single botnet into an enduring template, a "playbook" that would be reused for years to come.

Mirai a Decade Later: Aisuru and Kimwolf

The durability of that playbook is confirmed by the most recent botnets, which show that a decade of progress in the industry has not closed the underlying gaps. **Aisuru** is a Mirai-derived botnet first observed in late 2024, growing to encompass up to millions of routers, digital video recorders, and IP cameras. **Kimwolf**, which appeared in December 2025, infected more than two million Android TVs and set-top boxes, recruited through residential proxy networks and through an **exposed Android Debug Bridge** interface.

The scale of the disruption these botnets could inflict reached unprecedented levels. Record distributed-denial-of-service events included a **15.7-terabit-per-second attack against Microsoft Azure in October 2025**, followed by a **31.4-terabit-per-second attack, generating 14 billion packets per second, in November 2025**. Their **business model remained fundamentally unchanged since 2016: DDoS-for-hire and residential proxies**. In **March 2026, the United States Department of Justice disrupted Aisuru, Kimwolf, and the related botnets JackSkid and Mossad, together comprising more than three million devices**.

The lesson is stated plainly and it is the culmination of the entire Mirai narrative: **the Mirai playbook still works; the devices changed, but the vulnerabilities did not.** Ten years of new hardware, from IP cameras to Android TV boxes, has done nothing to change the fundamental weaknesses, default credentials, exposed services, absent updates, that the original Mirai exploited.

BadBox 2.0: Compromised Before the First Boot

A final recent case introduces a threat that even a perfectly diligent user cannot prevent: the **supply-chain compromise**. In the **BadBox 2.0** campaign of 2025, a **backdoor was pre-installed at the factory** in low-cost Android TV boxes, tablets, and projectors. The scale was enormous, affecting **about ten million devices**; the **FBI issued a public alert in June 2025**, and Germany's federal cybersecurity agency, the **BSI, had already sinkholed 30,000 of the devices in December 2024**. The infected devices were **used for ad fraud, residential proxying, and as a springboard for delivering further malware**.

What makes BadBox 2.0 conceptually distinct is that **no user action can prevent it**: the device ships already compromised, and the update channel itself belongs to the attacker, so there is no clean state to return to and no trusted mechanism through which to remediate. The lesson expands the threat model in a fundamental way: **supply-chain trust is part of the security picture, and OWASP IoT risk number five, the use of insecure or outdated components, begins before the device is even purchased.** Security can no longer be assumed to start at the moment of installation; it must be traced all the way back through the manufacturing and distribution chain.

Securing the Internet of Things

The Main Challenges of IoT Security

Before examining defensive measures, it is important to consolidate the reasons why securing the IoT is intrinsically difficult. These challenges are not incidental; they are structural features of the domain that any solution must contend with.

The first challenge is the **very large attack surface combined with widespread deployment**: with tens of billions of devices distributed across every corner of physical and digital life, the sheer number of entry points is unmanageable by conventional means. The second is the **limited resources of the devices themselves**: constrained energy budgets and modest computational power make it hard to run the cryptographic and monitoring machinery that security demands. The third is the **lack of standards and regulations**, which leaves the field fragmented and inconsistent. The fourth is the difficulty of **integrating safety and security processes**, since in cyber-physical systems the two concerns are entangled and must be reconciled rather than treated separately.

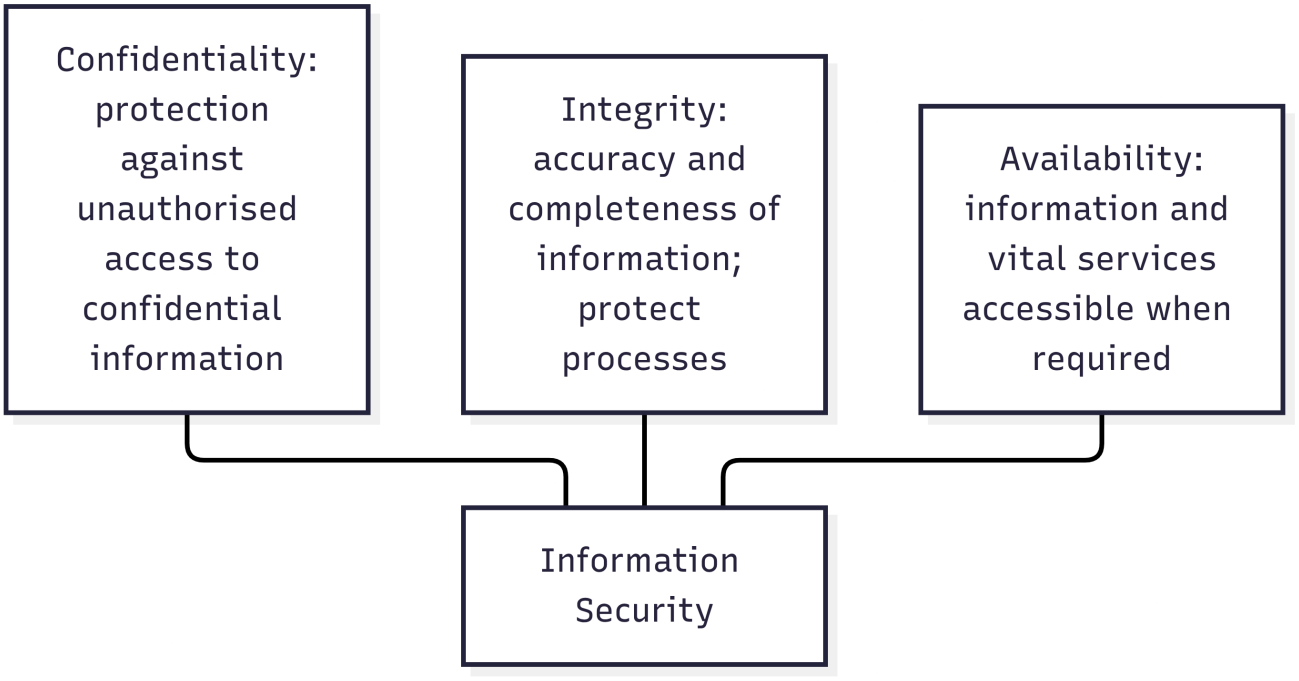
Further challenges compound these. **Security by design is not a top priority** for many manufacturers, who compete on price and features rather than

protection. There is a pervasive **lack of expertise**, as the developers of cheap connected devices are often not security specialists. **Applying security updates** is difficult or impossible once devices are fielded, as noted throughout the history of the domain. **Insecure development** practices introduce flaws at the source. And **unclear liabilities** mean that responsibility for a breach is often diffuse, so that no single party is fully accountable for securing the device across its life. Together these challenges explain why the same vulnerabilities recur decade after decade.

The CIA Triad

The foundational framework for reasoning about any security problem, including the IoT, is the **CIA triad**, which names the three core properties of information security: **Confidentiality, Integrity, and Availability**.

Confidentiality exists to ensure protection against unauthorised access to, or use of, confidential information. **Integrity** exists to ensure the accuracy and completeness of information, so as to protect the business processes that depend on it. **Availability** exists to ensure that information and vital services are accessible for use whenever they are required. These three properties form the vertices of a triangle at whose centre sits the overall goal of information security; a defence that satisfies all three is complete, while a weakness in any one of them compromises the whole.



The CIA Triad Applied to the IoT

When the triad is mapped onto the specific realities of the IoT, each property acquires a concrete and somewhat expanded meaning.

Confidentiality and privacy in the IoT require that the data collected from a device should not reveal secure information to any unauthorised device, and that data should remain controlled by the corresponding user to whom it belongs. Because IoT data is so often personal, privacy is inseparable from confidentiality here. Three families of technique are used to protect it. **Anonymity-based** approaches strip or obscure the identity associated with data so that even if it is seen, it cannot be linked to an individual. **Encryption-based** approaches render the data unreadable to anyone lacking the key. **Perturbation-based** approaches deliberately add noise or distortion to the data so that its aggregate statistical value is preserved while individual records are protected.

Integrity in the IoT means, straightforwardly, that the **data is not tampered with**, so that the readings a system acts upon and the commands it issues are exactly as intended and have not been altered by an attacker, which, as Stuxnet showed, is the difference between a plant that is safe and one that is being quietly destroyed.

Availability in the IoT means that the **services can always access the data** they need, and, critically, that they can do so in **real time**. The real-time requirement is a distinctive and demanding feature of the IoT: a delayed reading in a control loop for a vehicle brake or an insulin pump is not a minor degradation but a safety failure, which is why availability in this domain carries a weight it does not always carry in conventional IT.

Other Security Features

Beyond the three core CIA properties, two additional security features are essential in the IoT context.

The first is **identification and authentication**, which must be established for three distinct classes of entity: the **devices** themselves, the **applications** that interact with them, and the **users** who operate the system. Each must be reliably identified and its identity verified before it is granted access, because in an environment of billions of interacting components, the question of "who or what is this" is foundational to every other control.

The second is **trust**, which must be established along several dimensions: between the **layers** of the architecture, between the **devices**, and between the

applications. Because a smart environment is assembled from heterogeneous components made by different manufacturers and operated by different parties, the relationships between these components cannot be assumed to be trustworthy by default; trust must be deliberately established and maintained, at each layer boundary and between each pair of interacting entities.

The OWASP Top 10 IoT Risks and Vulnerabilities

The single most authoritative catalogue of the ways in which IoT devices go wrong is the list of the **ten most important IoT risks and vulnerabilities** published by OWASP (the Open Worldwide Application Security Project). This list is worth committing to memory, because nearly every real-world incident examined in this chapter, from Stuxnet's hardcoded WinCC password to the Aliquippa water authority's default `1111` to the Kia dealer portal, maps directly onto one of its ten entries. The list functions as the connective tissue of the whole subject: it is the vocabulary in which the recurring root causes are named.

The **first risk is weak, guessable, or hardcoded passwords**. This is the use of credentials that are easily brute-forced, publicly available, or simply unchangeable because they are baked into the device. It is the most common and most consequential IoT weakness, and it is the one that enabled both Mirai's brute-forcing of consumer devices and the CyberAv3ngers' takeover of water-utility PLCs.

The **second risk is insecure network services**. These are unneeded or insecure network services running on the device itself, especially those exposed to the Internet, which can compromise the confidentiality, integrity, or availability of information or allow unauthorised remote control. Telnet, the service Mirai scanned for on port 23, is the archetypal example of an insecure network service that should never have been exposed.

The **third risk is insecure ecosystem interfaces**. These are insecure web, back-end API, cloud, or mobile interfaces existing in the ecosystem *outside* of the device, which nonetheless allow the compromise of the device or its related components. The Kia case is the perfect illustration: the vulnerability lived entirely in the dealer web portal and its API, not in the car.

The **fourth risk is the lack of a secure update mechanism**. This is the absence of the ability to securely update the device, and its symptoms include the

lack of firmware validation on the device, the lack of secure delivery such as transmitting updates in plaintext, and the lack of anti-rollback mechanisms that would prevent an attacker from forcing a device back to a known-vulnerable version.

The **fifth risk is the use of insecure or outdated components**. This is the use of deprecated or insecure software components and libraries that could allow the device to be compromised, and it includes the insecure customisation of operating-system platforms and the use of vulnerable third-party software. The BadBox 2.0 supply-chain compromise is a manifestation of this risk operating even before purchase.

The **sixth risk is insufficient privacy protection**. This is the situation in which a user's personal information is stored on the device and is then used insecurely or without permission, a direct affront to the confidentiality-and-privacy property discussed earlier.

The **seventh risk is insecure data transfer and storage**. This is the lack of encryption or access control for sensitive data anywhere within the ecosystem, whether that data is at rest in storage, in transit across the network, or in use during processing. All three states of data must be protected, not merely one.

The **eighth risk is the lack of device management**. This is the absence of security support for devices deployed in production, encompassing asset management, update management, secure decommissioning, systems monitoring, and response capabilities. Without device management, an operator cannot even know what devices they have, let alone secure or retire them.

The **ninth risk is insecure default settings**. This describes devices or systems shipped with insecure default settings, or those that lack the ability to be made more secure because they restrict operators from modifying their configurations. A device that arrives insecure and cannot be hardened is insecure for its entire life.

The **tenth risk is the lack of physical hardening**. This is the absence of physical hardening measures, which allows potential attackers to gain sensitive information that can aid future remote attacks, or to take local control of the device. Because IoT devices live in the physical world, often in places accessible to attackers, physical access is a genuine and often underestimated threat.

The following table consolidates the list for study and reference.

#	Vulnerability / Risk	Description
1	Weak, Guessable, Hardcoded Passwords	Using easily brute-forced, publicly available, or unchangeable credentials
2	Insecure Network Services	Unneeded or insecure network services on the device, especially those exposed to the Internet, compromising the CIA of information or allowing unauthorised remote control
3	Insecure Ecosystem Interfaces	Insecure web, back-end API, cloud, or mobile interfaces outside the device that allow compromise of the device or its related components
4	Lack of Secure Update Mechanism	Inability to securely update the device: no firmware validation, plaintext delivery, no anti-rollback mechanisms
5	Use of Insecure or Outdated Components	Deprecated or insecure software components/libraries, insecure OS customisation, vulnerable third-party software
6	Insufficient Privacy Protection	User's personal information stored on the device and used insecurely or without permission
7	Insecure Data Transfer and Storage	Lack of encryption or access control for sensitive data at rest, in transit, or during processing
8	Lack of Device Management	No security support in production: asset management, update management, secure decommissioning, monitoring, response
9	Insecure Default Settings	Insecure default settings, or inability to harden the system because operators cannot modify configurations
10	Lack of Physical Hardening	No physical hardening, allowing attackers to extract sensitive information for future remote attacks or to take local control

Two Categories of Flaw: Design and Library

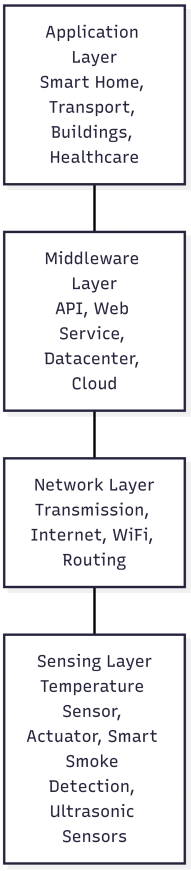
It is illuminating to observe that the failures catalogued by OWASP tend to originate in one of two broad categories. The first is the **design flaw**, a weakness rooted in the fundamental architecture of a product, such as an unpatchable flaw

in a modern car that allows an attacker to disable safety features; because the flaw is architectural, it often cannot be remediated after the fact. The second is the **library flaw**, a weakness introduced by a defective shared software component, such as the "Devil's Ivy" vulnerability, in which a single bad code library triggered a vulnerability across millions of IoT devices at once. The library category is particularly dangerous because a single defect propagates automatically into every product that incorporates the affected component, echoing directly the OWASP risk of insecure or outdated components.

Sources of Security Threats by Architectural Layer

Because an IoT application is built from several architectural layers, and because **each of these layers uses diverse technologies that bring their own issues and security threats**, it is analytically useful to catalogue the attacks that afflict each layer in turn. This layered view of the threat landscape complements the OWASP list: where OWASP names the categories of weakness, the per-layer catalogue locates the specific attacks at the point in the architecture where they occur.

For this purpose the architecture is conventionally divided into four layers stacked from the physical world upward: the **sensing layer** at the bottom, containing temperature sensors, actuators, smart-smoke detectors, and ultrasonic sensors; the **network layer** above it, handling transmission, Internet connectivity, Wi-Fi, and routing; the **middleware layer**, containing APIs, web services, the datacenter, and the cloud; and the **application layer** at the top, encompassing smart-home, smart-transport, smart-buildings, and smart-healthcare applications.



Security Issues at the Sensing Layer

The sensing layer, being physically exposed and resource-constrained, is subject to a broad range of attacks. **Node capturing** is the physical seizure of a sensor node by an attacker. **Malicious code injection** introduces attacker-controlled code into a node. **False data injection** feeds the system fabricated sensor readings, corrupting the information on which decisions are based. **Replay attacks** capture legitimate transmissions and retransmit them later to deceive the system. **Cryptanalysis and side-channel attacks** attempt to recover secret keys, either by breaking the cryptography directly or by observing physical characteristics such as timing or power consumption. **Eavesdropping and interference** intercept or disrupt the communications of the layer. **Sleep-deprivation attacks** exploit the energy constraints of battery-powered nodes by keeping them awake until their batteries are exhausted, a form of denial of service unique to constrained devices. And **booting vulnerabilities** exploit the moment of start-up, when a device may be in an insecure state before its protections are fully active.

Security Issues at the Network Layer

The network layer, responsible for moving data, is subject to attacks against communication and routing. **Phishing-site attacks** attempt to lure users or systems to fraudulent destinations. **Access attacks** seek unauthorised entry into

the network. **Denial-of-service and distributed-denial-of-service, (D)DoS, attacks** overwhelm the network to deny service. **Data-transit attacks** target information while it is moving across the network. **Routing attacks** subvert the paths that traffic follows, and two important sub-types are named explicitly: the **sinkhole attack**, in which a malicious node advertises itself as an attractive route in order to draw traffic through itself, and the **wormhole attack**, in which two colluding nodes tunnel traffic between distant points to distort the network's topology. Finally, **spoofing attacks** forge identities or addresses to impersonate legitimate participants.

Security Issues at the Middleware Layer

The middleware layer, which hosts the APIs, web services, and cloud back-end, is subject to attacks characteristic of web and cloud infrastructure. The **man-in-the-middle attack** interposes the attacker between two communicating parties. **SQL injection** exploits improperly handled input to manipulate a back-end database, exactly the class of attack Stuxnet used against the WinCC database. The **signature-wrapping attack** manipulates the digital signatures used in web-service messages to alter their content while preserving apparent validity. **Cloud malware injection** introduces malicious code or services into the cloud environment. And the **flooding attack in the cloud** overwhelms cloud resources to deny service or inflate costs.

Security Issues at the Gateways

The gateway, occupying the sensitive boundary between the device world and the wider network, presents its own distinct set of security concerns. **Secure onboarding** is the challenge of safely admitting a new device into the environment through the gateway without allowing an impostor to join. **Extra interfaces** are a concern because gateways often expose additional management or debugging interfaces that enlarge the attack surface. **End-to-end encryption** is a concern because, as noted earlier, the gateway may terminate and re-encrypt traffic, seeing it in clear at the point of translation, so ensuring genuinely end-to-end protection is difficult. And **firmware updates** are a concern because the gateway, like the devices it serves, must itself be kept current, yet is subject to the same update difficulties.

Security Issues at the Application Layer

The application layer, closest to the user and the business logic, is subject to attacks aimed at data and functionality. **Data thefts** target the sensitive information the application handles. **Access-control attacks** attempt to bypass the permissions that govern who may do what. **Service-interruption attacks** deny the application's availability to legitimate users. **Malicious code injection** introduces attacker code into the application. **Sniffing attacks** capture sensitive application data in transit. And **reprogram attacks** attempt to alter the application's behaviour by rewriting its logic, echoing at the application level the kind of logic subversion that Stuxnet performed at the controller level.

Making the IoT Reliable, Secure, and Robust

Having surveyed the threats layer by layer, the final concern is the constructive one: the practices and technologies through which an IoT system can be made genuinely reliable, secure, and robust. These measures span the full life of a device, from its development through its deployment to its eventual retirement, and they address the challenges enumerated earlier at their source.

Penetration testing subjects a system to simulated attacks in order to discover its weaknesses before a real attacker does. The **use of encryption at different layers** protects data throughout the architecture rather than at a single point, so that confidentiality and integrity are maintained end to end. **Authentication** ensures that only legitimate devices, applications, and users gain access. **Device test and validation** verifies that a device behaves correctly and securely before it is trusted. **Reducing security costs** matters because, given the extreme price sensitivity of IoT hardware, security measures must be affordable enough to be adopted at all. **Data-flow validation** checks that information moves through the system only along expected and legitimate paths.

Further measures address the operational and end-of-life phases. **Machine-learning-based security** brings adaptive detection of anomalies and attacks, particularly valuable given the scale and variety of IoT traffic. **Automatic firmware patching and updating** confronts head-on the perennial difficulty of keeping fielded devices current, removing the reliance on manual intervention that so often fails. **Notification** ensures that operators and users are informed of security-relevant events. **Facilitating device identification** supports the foundational requirement that every entity in the environment be reliably known. And **end-of-life** management ensures that devices are securely decommissioned when they are retired, so that they do not linger as unmonitored, unpatched

liabilities, the very "soft targets" that the Age of Protection was meant to eliminate.

Defensive Security Measures Catalogue

The defensive posture can be organised as a mapping from each category of weakness to the concrete security considerations that address it. This catalogue is the practical counterpart to the OWASP list of risks: for every way a device can be weak, it names what a responsible designer or operator should do. The following two tables consolidate the full set of measures.

Category	IoT Security Consideration
Insecure Web Interface	Disallow weak user passwords; provide an account-lockout mechanism; test the interface for SQL-injection, XSS, and CSRF vulnerabilities
Insufficient Authentication / Authorization	Require strong passwords for authentication; implement two-factor authentication; force password expiration after a certain date
Insecure Network Services	Ensure all devices operate with minimal ports active; ensure devices do not expose network ports or services to the Internet via UPnP; review required network services for vulnerabilities
Lack of Transport Encryption	Ensure traffic is encrypted between all system components; ensure SSL/TLS implementations are updated and configured properly
Privacy Concerns	Ensure only the minimal amount of personally identifiable information (PII) is collected from consumers; ensure only non-sensitive data is analysed; ensure a data-retention policy is in place

The measures continue with the interfaces and physical dimensions of the system.

Category	IoT Security Consideration
Insecure Cloud Interface	Ensure all cloud interfaces are reviewed for vulnerabilities; ensure any cloud-based web interface disallows weak passwords; ensure all cloud interfaces use transport encryption

Category	IoT Security Consideration
Insecure Mobile Interface	Ensure any mobile application disallows weak passwords; ensure any mobile application has an account-lockout mechanism; implement two-factor authentication for mobile applications
Insufficient Security Configurability	Ensure password-security options are made available (for example, enabling 20-character passwords or two-factor authentication); ensure encryption options are made available (for example, enabling AES-256); ensure secure logging is available for security events
Insecure Software / Firmware	Ensure all system devices have update capability and can be updated quickly when vulnerabilities are discovered; ensure update files are encrypted and are also transmitted using encryption
Poor Physical Security	Ensure the device is produced with a minimal number of physical external ports (for example, USB ports); ensure the firmware or operating system cannot be accessed through unintended methods such as an unnecessary USB port; ensure the product is tamper-resistant

Read together, these tables express the core philosophy of IoT defence. The recurring themes, disallowing weak passwords, providing lockout and two-factor mechanisms, minimising exposed ports and interfaces, encrypting data everywhere and in every state, enabling updates, minimising collected data, and hardening the device physically, are precisely the countermeasures to the OWASP Top 10 risks and to the layer-by-layer attacks catalogued above. The defensive story therefore closes the loop that the whole chapter has traced: the same small set of root causes appears again and again across every incident from Stuxnet to BadBox, and the same small set of disciplined measures, applied consistently across the device's entire life and across every layer and interface of the ecosystem, is what stands between a smart environment and the physical, personal harm that its compromise would cause.

The ultimate synthesis is the one with which the subject began. Because users live inside these systems, security failures become physical and privacy failures become personal; and because the devices are cheap, numerous, long-lived, and rarely updated, the discipline of securing them cannot be an afterthought bolted on after deployment but must be designed in from the start, sustained through operation, and honoured all the way to end-of-life.

CHAPTER 2: IoT part 2

Introduction to the Attestation Problem

The security of the Internet of Things rests on a problem that is deceptively simple to state yet remarkably difficult to solve: how can one party become confident that a remote device is running the software it is supposed to be running, and nothing else? In a smart environment populated by cameras, sensors, actuators, medical monitors, and industrial controllers, each device is a potential foothold for an attacker. A compromised device may continue to behave outwardly as expected while silently exfiltrating data, manipulating readings, or serving as a launching point for further intrusion. The family of techniques designed to detect such compromise is known collectively as **attestation**.

Attestation occupies a specific niche within the broader landscape of IoT security. A useful way to situate it is to recognize that IoT security concerns span three stacked layers, each with its own distinctive threats and countermeasures. At the top sits the **application layer**, which encompasses concrete IoT applications such as intelligent logistics, smart home systems, remote medical care, smart grids, intelligent traffic control, and environmental monitoring, together with the application-support infrastructure beneath them: middleware, service-support platforms, cloud computing platforms, and information-development platforms, each requiring its own security treatment. In the middle lies the **network layer**, which carries data across local area networks (where local area network security and technologies such as 3G security operate), across the core network (dominated by Internet security concerns), and across the access network (where ad hoc, GPRS, and WiFi security mechanisms apply). At the foundation rests the **perception layer**, the stratum of physical data acquisition. It is here, among the perception networks and perception nodes, that attestation finds its home. The perception layer must secure RFID technology, wireless sensor networks (WSN), and radio-sensor networks (RSN), addressing protocol security, base-station and reader security, tag counterfeit and tag-encoding security, routing-protocol security, cryptographic algorithms, key management, node-trust management, and the fusion of sensor, RFID, and WSN security. The physical, resource-constrained devices that populate this layer are precisely the devices whose internal state attestation seeks to verify.

This chapter develops the subject in three movements. It begins with **remote attestation** as a general concept and formal object, proceeds to **device attestation** in its software-based, hardware-based, and hybrid variants, and

culminates in **collective remote attestation**, which scales the problem up to entire networks and swarms of devices.

Remote Attestation

Remote attestation is, at its core, a protocol between two parties. The first is a **challenger**, conventionally denoted `Chal`, whose role is to verify the internal state of a second party. That second party is a device called the **prover**, denoted `Prov`, whose internal state is the object under scrutiny. The entire interaction is structured so that the challenger can reach a justified conclusion about whether the prover is in an acceptable condition.

The goal of the protocol is to allow an honest prover—one that has not been compromised—to create an **authentication token** that convinces the challenger that the prover is in some well-defined, expected state. The expected state is typically a specific configuration of memory contents: the firmware, application code, and critical data that the device is supposed to be running. When the prover is genuinely in that state, it must be able to produce a token that the challenger will accept.

The protocol's security property is the mirror image of its functional goal. If the prover has been compromised and its internal state has been modified—for instance, if malware has been injected into its memory—then the authentication token must reflect this fact. In other words, a compromised prover must be unable to produce a token that falsely convinces the challenger that everything is normal. The entire value of attestation collapses if a compromised device can forge an acceptable token, so the design of the token-generation mechanism is where the intellectual difficulty concentrates.

The Adversary Model

To reason rigorously about attestation, one must first be precise about what the attacker is assumed to be capable of. The adversaries relevant to IoT attestation divide naturally into two broad families: **software-based** adversaries and **hardware-based** (physical) adversaries. Cutting across both families are two orthogonal capabilities that any serious adversary may possess.

The first such cross-cutting capability concerns **communication**. A communication adversary is assumed to have full control of the communication channel. This control is expansive: it includes eavesdropping on all traffic, injecting packets of the attacker's own construction, modifying packets in transit, dropping

packets entirely, and delaying their delivery. An adversary with this degree of channel control can manipulate the timing and content of every message exchanged during an attestation protocol, which has profound implications for protocols that rely on timing guarantees.

The second cross-cutting capability is **mobility**. A mobile adversary is one that is able to erase any trace of its presence. This capability matters because it undermines forensic approaches that would otherwise detect compromise after the fact; if the attacker can clean up after itself, then detection must happen during the attestation itself rather than through later inspection of logs or residual artifacts.

Within the **software-based** family, three distinct attack flavors appear, each with its own purpose. The **remote** software attack aims to infect one or more devices with malware, operating entirely from a distance without any physical access. The **local** software attack has the purpose of impersonating devices, standing in for a legitimate device in protocol exchanges. The **physical non-intrusive** attack, despite involving physical proximity, is grouped with software attacks because it does not alter the device's internals; its purpose is to learn a device's secret, to impersonate or clone the device, and potentially to launch side-channel attacks that extract information by observing physical emanations such as timing, power consumption, or electromagnetic radiation.

Within the **hardware (physical)** family, two more aggressive attack flavors appear, distinguished by how deeply they violate the device's physical integrity. The **stealthy physical intrusive** attack has the purpose of capturing a device and physically extracting its secrets, then cloning the device—all while attempting to remain undetected. The **physical intrusive** attack goes further still: its purpose is to capture a device and actively modify its contents or components, rewiring or reprogramming the hardware itself. The progression from software-only to stealthy-physical to fully-physical-intrusive represents an escalating ladder of attacker power, and attestation schemes are generally characterized by how far up this ladder they can defend. (This taxonomy of adversaries follows the framework articulated in the work on building trust in IoT systems by Abera and colleagues.)

Device Attestation: The Software-Based Approach (SWATT)

The earliest and conceptually simplest form of device attestation is purely software-based, requiring no specialized hardware on the device. The canonical

example is **SWATT** (Software-based ATTestation), introduced by Seshadri, Perrig, Van Doorn, and Khosla in 2004. The scenario it addresses is one in which a **prover**—imagine a networked surveillance camera running some untrusted software—must convince a **verifier**, a more capable machine such as a server, that its memory contents are exactly what they should be.

The main idea of the software-based approach rests on three pillars. First, the adversary is assumed to be a **software-only attacker**, meaning the defenses need not withstand physical tampering. Second, the scheme makes **no use of specialized hardware**, which is precisely what makes it attractive for cheap, low-end devices. Third, the scheme typically relies on a **challenge-response protocol**: the verifier sends a challenge, the prover computes a response that depends on both the challenge and its memory contents, and the verifier checks the response.

The approach rests on two explicit assumptions about the setup. The device must contain a **memory-content-verification procedure** that the verifier can activate remotely—that is, there must be code on the device whose job is to read through memory and compute a summary of what it finds. And the verifier must know the **exact hardware architecture and the expected memory contents** of the device. This second assumption is what gives the verifier the ability to predict, in advance, what the correct response to any given challenge should be. Because the verifier possesses an exact replica of the expected device state, it can compute the expected response independently and compare it against what the prover actually returns.

Formalizing Remote Attestation

Beyond the intuitive description, remote attestation admits a precise formal definition, which clarifies exactly what guarantees are being sought. Following the minimalist formulation of Francillon, Nguyen, Rasmussen, and Tsudik, an attestation protocol **P** is composed of three algorithms.

The first is **Setup(1^n)**, a *probabilistic* algorithm that takes as input a security parameter 1^n (the superscript n expressed in unary, which is the standard cryptographic convention for parameterizing security) and outputs a **long-term key k**. This key is the shared secret or key material that underpins the trust relationship between the parties. Because it is probabilistic, each invocation can produce fresh key material.

The second algorithm is **Attest(k; s)**, a *deterministic* algorithm. Given the key **k** and a device state **s**—where the state is structured as the pair s = (s_chal, s_prov), capturing the relevant state of both the challenger and the prover—it outputs an **attestation token α**. Determinism here is essential: for a fixed key and a fixed state, the token is always the same, which is what allows the verifier to predict it.

The third algorithm is **Verify(k; s; α)**, also *deterministic*. Given the key **k**, a device state **s**, and an attestation token **α**, it outputs **1 if and only if α corresponds to s**—that is, if and only if Attest(k; s) = α—and outputs **0 otherwise**. Verification thus reduces to recomputing the expected token and checking for an exact match.

The security of this construction is captured by a single inequality concerning forgery. The probability that an adversary wins the attestation-forgery game, written Pr[Att-Forgery_{chal,prov}(n) = 1], must be less than or equal to **negl(n)**, a *negligible* function of the security parameter. A negligible function is one that shrinks faster than the inverse of any polynomial, meaning that as the security parameter grows, the attacker's chance of producing a token that is accepted despite a compromised state becomes vanishingly small. This inequality is the formal expression of the earlier intuitive requirement that a compromised prover must not be able to forge an acceptable token.

The Properties a Secure Scheme Must Enforce

For the formal guarantees to hold in practice, the attestation mechanism must enforce a set of concrete properties on the device. These properties are what prevent a clever attacker from subverting the computation of the token.

The first property is **exclusive access to k**: only the attestation routine may read or use the long-term key, and no other code on the device may touch it. If arbitrary code could read the key, then malware could compute valid tokens on behalf of a compromised state. The second property is that there must be **no leaks of k other than α**: the only thing the attestation process reveals to the outside world is the token itself, and the token must not inadvertently disclose the key. The third property is **code immutability**: the attestation code itself cannot be modified by an attacker, because if the verification routine could be rewritten, it could simply be made to lie.

Two further properties govern the execution environment rather than the key. **Uninterruptible execution** demands that once the attestation routine begins, it

runs to completion without being interrupted—an interrupt handler controlled by malware could otherwise seize control mid-computation, inspect intermediate values, or substitute results. **Controlled invocation** requires that the attestation code can only be entered at its proper entry point and executed as a coherent whole, rather than being called into at arbitrary offsets where an attacker might skip critical checks.

How an Attacker Tries to Evade Software Attestation

The central intuition behind software attestation's security—in the absence of any trusted hardware—is a clever argument about *overhead*. The reasoning runs as follows. Suppose an attacker has replaced the legitimate firmware with malicious code but has kept a copy of the original, correct code elsewhere in memory so that it can still produce correct answers when challenged. To pass the attestation, this **malicious prover needs to redirect the attestation challenge to the trusted (original) software** to obtain the proper result. The key claim is that **the overhead incurred by this memory redirection will be noticed**—the extra indirection costs time, and if the verifier measures how long the response takes, the delay betrays the attack.

This argument becomes concrete when one compares the two memory layouts. In the **expected memory layout** of an honest device, memory is organized into three contiguous regions: a block of **verification code**, followed by the **firmware**, followed by an **empty** region. The verifier knows this layout precisely and expects the attestation to sweep across exactly these contents.

In the **attacker's memory layout**, by contrast, the organization is subtly different. The region that should hold the verification code instead holds **malicious code**. The firmware region remains, but crucially, the previously empty region now holds a copy of the **old verification code**—the attacker has stashed the legitimate code there so it can be consulted to generate correct responses. For the attacker to produce the right answer, its malicious code must detect when memory is being read for attestation purposes and redirect those reads to the stashed copy of the legitimate code, so that the checksum comes out correct even though the live verification region has been corrupted. It is this redirection, performed on potentially every memory access during the attestation sweep, that imposes measurable overhead and, in principle, reveals the deception.

The Verification Procedure

Having established *why* memory redirection is detectable in principle, the question becomes *how* to design the verification procedure so that the overhead is both unavoidable and conspicuous. The design hinges on forcing the attestation routine to read memory in an unpredictable, exhaustive manner that leaves no room for shortcuts.

The mechanism begins with a **nonce**—a fresh random value supplied by the verifier with each challenge. This nonce is used as the **seed to a pseudo-random generator (PRG)**, and the PRG in turn generates the sequence of memory addresses that the verification routine will access. Because the addresses are derived pseudo-randomly from a nonce the attacker cannot predict in advance, the attacker cannot know ahead of time which locations will be touched or in what order. The access pattern, visualized, resembles a chaotic scribble of arrows crisscrossing the entire memory space, jumping unpredictably from region to region rather than marching through memory in any orderly sweep.

Two consequences follow from using a pseudo-random traversal. First, the procedure will **touch every memory location with high probability**. The relevant bound is **O(n log n)**: this is the classical coupon-collector result, which says that to cover all n locations with high probability when sampling them at random, roughly $n \log n$ samples are required. By running the traversal for this many iterations, the designer ensures that no region of memory escapes inspection, so an attacker cannot hide malicious code in some corner that the checksum happens to skip. Second, the verification code itself must have a **small code size**, both so that it fits on constrained devices and so that there is little room for an attacker to hide alterations within it.

The design also builds in a specific response to tampering. If memory has been altered by the attacker, then whenever the traversal reads one of the altered locations, the attacker's code must substitute the **original value** that *should* be there, because returning the modified value would produce a wrong checksum. This substitution is exactly where the overhead is incurred. Concretely, the attacker's code must insert a conditional check—an **if instruction**—before each memory read, to decide whether the address being read is one of the tampered locations (in which case the stashed original must be returned) or an untouched location (which can be read directly). Every such conditional adds instructions to the inner loop, producing a cumulative **slowdown** across the millions of iterations in a full attestation.

Three further properties reinforce the scheme's resistance to forgery. There is **no replay attack**, because the fresh nonce guarantees that the challenge—and therefore the expected response—differs every time; a response recorded from a previous session is useless for a new one. The checksum computation is deliberately engineered to be **not parallelizable**, meaning the attacker cannot distribute the work across multiple cooperating devices to recover the time lost to redirection—each step of the computation depends on the previous one, forcing strictly sequential execution. And there is **no precomputation**: because the response depends on the unpredictable nonce, the attacker cannot compute acceptable responses in advance during idle time.

Checksum Design and Implementation

The concrete realization of these principles is a carefully constructed **checksum** routine. The checksum is chosen to have a size of **n = 64 bits**, a length selected to make the probability of a **collision**—two different memory states producing the same checksum—extremely low. If the checksum were too short, a compromised state might accidentally or deliberately hash to the same value as the legitimate state, allowing the attacker to slip through; sixty-four bits makes such a coincidence negligibly unlikely. The checksum is maintained as a structured value composed of eight-bit cells, which the implementation updates one at a time as the traversal proceeds.

The structure of the computation can be read directly from its pseudocode. The checksum index **j** is initialized to zero. The pseudo-random generator is then "warmed up" by discarding its first 256 outputs (the values PRG_0 through PRG_{255}), a standard precaution with stream-cipher-style generators whose earliest outputs may exhibit statistical weaknesses. The checksum **C** is then initialized from the next eight generator outputs, PRG_{256} through PRG_{263}, giving it a pseudo-random starting value rather than a fixed constant.

The heart of the routine is a loop running over the index **i** from 265 to m + 265, where *m* governs the number of memory accesses. Each iteration performs several tightly coupled operations. First, it **constructs the address for the memory read** by combining a fresh generator output with feedback from the checksum itself: the address is formed as PRG_i shifted left by eight bits and added to a byte of the current checksum, C_{j−1 mod 8}. This dependence of the address on the current checksum state is crucial, because it chains the accesses together—the location read next depends on everything read so far, which is one of the mechanisms enforcing non-parallelizability.

The routine then **updates the checksum byte**. The relevant cell is recomputed as C*j* = *C_j + (mem[addr] xor C*{j−2 mod 8} + PRG_{i-1}): the value actually found in memory at the computed address is mixed, via exclusive-or and addition, with another checksum byte and a generator output. This is the step at which the *content* of memory enters the checksum, so any discrepancy between the real memory and the expected memory propagates into the final value. Immediately afterward, the updated byte is passed through a **rotate_left_one_bit** operation, a cheap diffusion step that spreads the influence of each input across bit positions and prevents simple cancellations. Finally the index is advanced, j = j + 1 mod 8, cycling through the eight checksum cells, and once the loop completes, the finished checksum **C** is returned.

The significance of this construction is that the address generation, the memory read, and the checksum update are **interlocked**: the next address depends on the checksum, and the checksum depends on the memory read at the current address. An attacker who wishes to feed false memory values into this pipeline must intercept each read and substitute the correct value, and must do so without disturbing the delicate chain of dependencies—an intervention that, as established, costs measurable time.

Assembly-Level Cost of Tampering

The abstract claim that an attacker's redirection "adds an if and slows things down" can be made quantitative by descending to the assembly level and examining a single cycle of the checksum loop. On a representative microcontroller architecture, one iteration of the legitimate routine decomposes into a sequence of primitive operations, each a single machine instruction.

The cycle begins by **generating the i-th member of the random sequence using an RC4-style generator**. This stage loads a constant into a register, reads a byte from one pointer with post-increment, performs a series of additions and loads that implement the stream generator's internal state update, and stores results back—a handful of load, add, and store instructions that together produce the next pseudo-random byte. The routine then **generates the sixteen-bit memory address** from that output with a register move. Next it **loads a byte from memory and computes a transformation**: it loads the program byte at the computed address, exclusive-ors it against a register holding checksum state, and adds in another register. Finally it **incorporates the output of this transformation into the checksum**: it adds the transformed value into the checksum register, shifts the checksum register left by one bit, adds back the carry

bit (the rotate operation realized as a shift plus carry-add), and moves a register to prepare for the next iteration.

The economy of this loop is the whole point. Because the legitimate cycle is already minimal, there is no "slack" the attacker can exploit. When the attacker inserts the conditional needed to redirect tampered addresses—the **if**—it costs **three additional instructions** per iteration. Measured against the tight baseline loop, those three instructions translate into roughly a **13% overhead**. Over a full attestation of hundreds of thousands of memory accesses, this percentage compounds into a time difference the verifier can reliably detect.

The empirical behavior confirms the analysis. When one plots execution time against the number of memory accesses, three lines emerge. The **legitimate verification code** traces a straight line rising steadily with the number of accesses. The **attacker's verification code**, burdened by the extra conditional, traces a parallel but visibly steeper line lying above it. And the **time difference** between them—plotted as a third line near the bottom—grows roughly linearly with the number of accesses, starting near zero and widening continuously. This growing gap is the verifier's signal: by setting a sufficiently large number of iterations, the honest-versus-malicious time difference can be pushed well beyond the noise floor of measurement and network jitter, making the attack detectable.

Avoiding Device Attestation: The Full Challenge-Response Protocol

The pieces assembled so far can now be composed into the complete software-based attestation protocol, which makes vivid both how legitimate attestation proceeds and how a physically relocated attacker attempts to subvert it. The central tension remains the same: a malicious prover must redirect the attestation challenge to trusted software to obtain the proper result, and the overhead of that redirection is what the scheme aims to expose.

Consider a scenario with a verifier and two provers. **Prover A** is the device under test, but it has been compromised—its live software is untrusted, and it is in collusion with **Prover B**, a separate device that still holds a copy of the correct, trusted software. The attack strategy is for Prover A to secretly forward the attestation work to Prover B, which can compute the honest answer, and then relay that answer back to the verifier as if it had computed it locally.

The protocol unfolds in four numbered steps. In **step 1**, the verifier generates a random nonce **N** and sends it to the prover. In **step 2**, the honest computation that is *supposed* to happen is the hashing of the device's own software: h = **HASH**(Software). On the trusted device (Prover B), this produces the correct hash **h** of the correct software. In **step 3**, the prover computes an authentication token δ = **MAC**__**k**(N ‖ h)—a message authentication code, keyed with the shared secret **k**, taken over the concatenation of the nonce and the software hash—and sends δ back to the verifier. In **step 4**, the verifier independently computes what the token should be, δ′ = **MAC**__**k**(N ‖ h′), using the hash **h′** of the configuration it *expects* the device to be running, and then checks whether **δ′ = δ**. If they match, the device is deemed to be in the expected state.

The use of a keyed MAC rather than a bare hash is what binds the response to both the fresh nonce (defeating replay) and the secret key (ensuring the attacker cannot simply compute the MAC without possessing **k**). The collusion attack by Prover A succeeds in producing a numerically correct δ only if the redirection to Prover B can be hidden—and in the software-only model the redirection's latency is exactly what the timing analysis of the preceding sections is designed to unmask.

The Limitations of Software-Based Attestation

For all its elegance, software-based attestation purchases its hardware-free simplicity at the cost of several serious limitations, each of which motivates the move toward hardware assistance in later sections.

The first limitation is that the entire security argument holds only against **software-only attackers**; it offers **no protection against a physical attacker**. The whole edifice rests on timing, and timing guarantees evaporate if an adversary can physically augment the device—for example, by attaching a faster processor or additional memory—so that the redirection overhead is absorbed by superior hardware rather than paid in observable time.

The second limitation is the reliance on a **strict time-bound response**. The verifier must know, with fine precision, how long an honest attestation should take, so that it can flag a response that arrives too late. Establishing this bound requires detailed knowledge of the device's exact clock speed and architecture, and leaves little tolerance for variability.

The third limitation follows directly from the second: the approach is **not feasible for multi-hop distances**. When the prover is several network hops away from

the verifier, the round-trip time becomes dominated by unpredictable network latency and queuing delays, which swamp the small computational overhead the scheme is trying to measure. The timing signal that distinguishes an honest prover from a redirecting attacker is simply lost in the noise of the network. This fragility over realistic network topologies is one of the strongest arguments for grounding attestation in something more robust than execution timing—namely, trusted hardware, the subject toward which the discussion now turns.

SCUBA: Secure Code Update By Attestation

Detecting that a device has been compromised is only half of a useful security story; the other half is doing something about it. **SCUBA**—Secure Code Update By Attestation—addresses this second half. Introduced by Seshadri, Luk, Perrig, Van Doorn, and Khosla, its goal is to **securely recover a device in the presence of malicious code**. That is, SCUBA assumes that a device may already be infected, and it provides a procedure by which a trusted base station can restore the device to a known-good state—patching out the malware—even though the malware is actively present and may try to resist.

The system model rests on a set of assumptions about the devices and the infrastructure. **Every node is identified by a unique ID embedded in a ROM**, so that identity is rooted in immutable read-only memory rather than in software that an attacker could rewrite. The **base station has complete knowledge of the devices**, meaning it holds an exact picture of each node's expected hardware and memory contents—the same prerequisite that made verification possible in the software-based setting. **Nodes know the public key of the base station**, which allows them to verify messages genuinely originating from the base station; here it is noted that public-key cryptography on such constrained hardware is costly, taking on the order of tens of seconds on low-power devices, which is why the design uses it sparingly. Finally, the **base station and the sensor nodes share a cryptographic key**, providing an efficient symmetric channel for the frequent operations where public-key operations would be too slow.

The adversarial assumptions complete the model. The attacker **can compromise sensor nodes by injecting malicious code**, and once a node is compromised, the **base-device shared key can be accessed** by the attacker—a sobering admission that the symmetric key alone cannot be trusted to prove a node's integrity, since the malware may have stolen it. The **base station itself is not compromised**; it is the anchor of trust. **Corrupted nodes can collude**, pooling their resources and coordinating their deceptions. There is, however, one essential

restriction on the attacker's power: the **attacker's own hardware devices are not present in the sensor network for the duration of the repair process**. This assumption is what makes recovery tractable—during the critical window of repair, the attacker cannot wheel in a faster machine to help a compromised node fake its responses, so the timing-based guarantees can hold.

The TOCTTOU Attack

Before SCUBA's recovery mechanism can be trusted, a subtle but devastating gap in naive attestation must be confronted. This is the **TOCTTOU** attack, an acronym for **Time-Of-Check-To-Time-Of-Use**. The attack exploits the interval between two distinct moments: the moment when code is verified, and the later moment when that same code is actually invoked for execution.

The problem is this: attestation, as described so far, checks what is in memory *at the time of the check*. But an attacker can **modify the code between the time the code is verified and the time the code is invoked for execution**. A device might pass its attestation honestly—its memory genuinely contains the correct code at the instant of measurement—and then, in the brief window before that code runs, the malware swaps in a malicious version. The verifier would have been shown a truthful snapshot of a clean state, yet the device would execute corrupted code moments later.

The consequence is stated starkly: **attestation cannot guarantee untampered code execution**. Verifying the contents of memory is simply not the same as guaranteeing that those verified contents are what actually run. This realization is pivotal, because secure recovery depends on being able to run recovery code that the attacker cannot subvert. If the very act of running the recovery routine opens a TOCTTOU window, then the malware could allow the recovery code to be verified and then hijack execution before the repair takes effect. SCUBA therefore needs a stronger primitive—one that binds verification and execution together so tightly that no window exists between them.

ICE: Indisputable Code Execution

That stronger primitive is **ICE**—Indisputable Code Execution—the engine at the heart of SCUBA. ICE is designed to deliver what plain attestation cannot: a guarantee not merely that the right code is present, but that the right code actually *executes*, in an environment the attacker cannot tamper with. Achieving **untampered code execution** proceeds in three stages.

The first stage is to **check the integrity of the executable**. This ensures that the executable has not been modified before it is invoked for execution—the familiar attestation check that confirms the code in memory matches the expectation. On its own this would still be vulnerable to TOCTTOU, which is why the remaining stages are essential.

The second stage is to **set up an untampered execution environment**, and it is this stage that closes the TOCTTOU gap. Two guarantees define this environment. First, the **execution of the executable is guaranteed to be atomic**: once begun, it completes as an indivisible unit. Second, **no other code on the sensor node is allowed to execute until the executable exits**. Together these guarantees eliminate the window the TOCTTOU attack relied upon—because nothing else can run between the integrity check and the execution, and because the execution cannot be interrupted and resumed, there is no moment at which malware could slip in a substitution. Verification and execution become welded into a single uninterruptible transaction.

The third stage is to **execute ICE and any other needed executable**. Crucially, **ICE is self-checksumming**: the ICE routine computes a checksum over itself (and over the code it is about to protect), so that any modification to the ICE code or its protected payload disturbs the checksum. This self-referential property is what makes ICE "indisputable"—the routine vouches for its own integrity as part of the very computation it performs, so an attacker cannot modify the routine without the modification showing up in the result. Once ICE has established this trustworthy bubble of execution, the SCUBA recovery code can run inside it with confidence that it will execute faithfully to completion.

Attacks Against Self-Checksumming

Because ICE's guarantee rests entirely on the integrity of its self-checksumming routine, the security analysis must anticipate every way an attacker might try to produce a correct checksum while actually running malicious code. The attacker's overarching goal in all of these is the same: to **fake the correct checksum within the expected time**. The defenses fall into two broad categories—**checksum forgery attacks**, which try to make a wrong state look right, and **speed-based attacks**, which try to buy back the time that cheating costs. Each attack is paired with the specific design countermeasure that neutralizes it.

Checksum Forgery Attacks

The **precomputation and replay attack** attempts to compute the checksum over the memory region containing ICE and the target executable *before* making any changes to that memory region. The attacker would compute the right answer while the memory is still clean, store it, then corrupt the memory and simply replay the stored answer. The countermeasure is that the **verifier sends a random challenge with every request**, and this challenge is fed into the checksum function. Because the challenge is unpredictable, no checksum computed in advance can be valid for a future request—precomputation is rendered useless.

The **data substitution attack** attempts to change some locations of the memory region containing the ICE verification function and the executable while keeping the original values at a different location in memory, so that reads can be redirected to the pristine copy. The countermeasure is that the **ICE verification function accesses memory in a pseudorandom pattern**, the same technique encountered with SWATT: unpredictable access ordering forces the attacker to insert per-access conditionals to redirect the reads, incurring detectable overhead.

The **memory copy attack (1)** attempts to compute the checksum over a copy of the correct code: the correct code is copied to another location in memory while the malicious code executes at the location where the correct code is supposed to reside. The defense is to **ensure that the contents over which the checksum is computed are fetched from the correct address locations in memory**. By weaving the actual addresses into the checksum computation, a copy stored at the wrong address cannot produce the right result.

The **memory copy attack (2)** is the mirror image: the correct code resides at the correct memory location, but the *malicious* code executes at another location of memory, so that the checksum sweeps the genuine code while control actually rests with the malware elsewhere. The countermeasure is to **ensure that the program counter is pointing to the correct memory addresses**, achieved by **including the program counter in the checksum**. If the PC is folded into the computation, then code running from the wrong address produces a different checksum, exposing the displacement.

The **forging CPU state inputs** attack attempts to forge the creation of the untampered execution environment by incorrectly setting up the CPU state and then forging the correct CPU state during checksum computation—faking the atomicity and isolation guarantees rather than genuinely providing them. The countermeasure **guarantees atomic execution by setting up the corresponding CPU state and incorporating that state into the**

checksum. Specifically, by including the **interrupt-disable bit** and the **CPU's interrupt vector table** in the checksum, the scheme ensures that the checksum is correct *only if* these inputs are correct—meaning only if interrupts are genuinely disabled and the interrupt handlers have not been tampered with. The very conditions that make execution untamperable are thereby made prerequisites for a passing checksum.

Speed-Based Attacks

The second category of attacks does not try to fake the checksum directly but rather to recover the time that any cheating would cost, so that the forgery can hide within the expected time window.

The **optimized implementation attack** attempts to decrease the execution time of the ICE verification function by optimizing the code, which would let the attacker use the time gained to forge the checksum without being detected. The countermeasure is to **keep the assembly code minimal**, using automated tools to either exhaustively find the most efficient implementation or to apply theorem-proving techniques to show that a given code fragment is provably optimal. If the legitimate implementation is already the fastest possible, the attacker has no faster version to run, so there is no time to steal.

The **multiple colluding devices attack** seeks to speed up execution by leveraging multiple devices to compute the checksum in parallel—several devices might collude to each compute a different range of the ICE verification loop and then combine their partial results into the final checksum. The countermeasure is to make the **verification function non-parallelizable**, forcing sequential execution. By chaining each step's input to the previous step's output, the computation cannot be split across cooperating devices, so additional hardware yields no speedup.

The **proxy attack** attempts to forward the ICE challenge to a proxy node with greater computing resources; the proxy holds a copy of the correct memory contents of the sensor node and computes the answer faster. Three defenses converge here. The scheme assumes that **all sensor nodes in the network have identical computing resources**, so no node can serve as a faster proxy; it assumes **the attacker is not physically present during verification**, so no external fast machine is available; and it relies on a **firewall in the base station that blocks packets during the execution**, so the challenge cannot be relayed out and an answer relayed back in during the critical window.

The **impersonation attack** has a legitimate sensor node impersonate the target in the ICE protocol by forwarding protocol messages from the base station to that legitimate node. The countermeasure is to **include the node ID in the computation**, so that each node's checksum is bound to its own identity and cannot stand in for another; it is also noted that proxy attacks of this kind require additional time, which the timing bound already penalizes.

Experimental Validation of ICE

The credibility of the entire timing argument rests on whether, in practice, the honest computation is reliably faster than the best possible cheating computation. This is tested by running ICE against the *fastest known* memory-copy attack over 40,000 iterations and recording the times across repeated samples.

The resulting measurements separate into clearly ordered bands. At the bottom lies the **legitimate code's runtime**—the pure computation time of the honest routine—forming the fastest and most stable trace. Slightly above it sits the **legitimate code's runtime plus network round-trip time (RTT)**, which adds the communication delay to the honest computation; this remains low, oscillating gently around a value a little below 2.9 seconds. Above these honest bands lies a dotted horizontal line marking the **expected time to compute the checksum**, the threshold the verifier uses to decide acceptance—positioned just above the honest traces to accommodate legitimate variation. Critically, both adversary traces lie *above* this threshold: the **adversary's runtime** sits at a flat band near 2.95 seconds, and the **adversary's runtime plus RTT** oscillates higher still, around 2.97–2.99 seconds. Because even the adversary's bare runtime exceeds the expected-time threshold, the cheating attacker cannot fit within the acceptance window, and the verifier distinguishes honest from malicious execution. The experiment thus empirically confirms the central claim: the overhead of the best available attack cannot be hidden within the time an honest device requires.

The Complete SCUBA Protocol

With ICE established as a trustworthy execution primitive and its resistance to forgery validated, the full SCUBA recovery protocol can be assembled. It is a timed challenge-response exchange between the **base station** and a **sensor node**, structured so that the base station first confirms the node is executing honest code and then delivers the precise patches needed to heal it.

The protocol runs as an ordered sequence of messages and local actions. The base station begins by recording **T1 = current time** and sending an **ICE challenge** to the sensor node. Upon receiving the challenge, the node **computes the ICE checksum over the memory region containing the ROM, the ICE verification function, and the SCUBA executable**—that is, it checksums exactly the code that must be intact for recovery to proceed, including its own identity-bearing ROM and the recovery routine itself. The node returns the **ICE checksum** to the base station.

The base station then records **T2 = current time** and performs two verifications. It checks that **(T2 − T1) < Time allowed**, confirming that the response arrived within the strict timing bound that rules out redirection and proxying, and it verifies the **ICE checksum** itself against the expected value. Only if both checks pass does the base station trust that the node has genuinely executed the honest ICE-protected code.

Trust established, the recovery phase proceeds. The node sends a **hash of its code memory** to the base station. The base station uses this **hash from the node to determine whether the node's code memory is modified**—comparing it against the known-good configuration to identify exactly what has been corrupted—and then **prepares code patches for the sensor node**, computing the minimal set of changes needed to restore it. These **code patches** are sent to the node, which finally **applies the patches**, overwriting the malicious modifications and returning to a known-good state. Because the patch application happens within the trustworthy window that ICE guarantees, the malware cannot interfere with its own removal.

Hardware-Based Attestation

Software-based attestation, however ingenious, is built on a foundation of sand: its guarantees rest on timing, and timing crumbles against physical attackers, multi-hop networks, and any adversary with faster hardware. **Hardware-based attestation** replaces this fragile foundation with a concrete one. Rather than inferring integrity from how *long* a computation takes, it anchors integrity in a dedicated, tamper-resistant hardware component that holds secrets the untrusted software can never reach and performs cryptographic operations that software cannot forge. The canonical embodiment of this approach is the architecture proposed by IBM, built around a **Trusted Platform Module (TPM)**.

The overall protocol follows the familiar challenge-response shape, but with the trusted hardware doing the cryptographic heavy lifting. Consider a **prover**—again, imagine a networked camera—facing a **verifier**. Inside the prover sits an **Attestation Service** running in cooperation with a **TPM**, and within the TPM reside three protected elements: **secure code**, one or more **PCRs** (Platform Configuration Registers), and cryptographic **keys**.

The exchange proceeds in three numbered steps. In **step 1**, the verifier generates a nonce **N** and sends it to the prover. In **step 2**, the nonce is passed into the TPM, where the attestation service draws on the PCR contents—which record the measured state of the platform—together with the nonce. In **step 3**, the TPM produces the attestation token $\delta = \mathbf{SIG_AIK(PCR} \parallel \mathbf{N)}$: a *digital signature*, computed with the **Attestation Identity Key (AIK)**, over the concatenation of the PCR values and the nonce. This signed token is sent to the verifier. The verifier then checks it by computing $\delta' == (\mathbf{SIG}_{\boldsymbol{AIK(PCR} \parallel \boldsymbol{N))}}$ **{AIK_PUB}**—that is, it verifies the signature using the **public AIK**—and tests whether $\delta' = \delta$.

The shift from a MAC to a *digital signature* is the quiet but important change here. In the software and hybrid schemes a symmetric MAC sufficed, which required the verifier to share the secret key. With a signature produced by the TPM's private AIK and verifiable by anyone holding the public AIK, attestation results become **publicly verifiable**: any party can check the token without possessing a secret, a property that becomes decisive when attestation scales to networks.

Beyond this core attestation flow, the hardware-based paradigm encompasses a broader family of trusted-computing schemes. These include mechanisms for **platform integrity**, realized through **secure boot** and **authenticated boot**; mechanisms for **secure storage**, which protect data at rest using hardware-held keys; and mechanisms for **isolated execution**, chief among them the **Trusted Execution Environment (TEE)**, which carves out a protected enclave in which sensitive code can run shielded from the rest of the system. These primitives recur throughout the remainder of the chapter as building blocks for more advanced architectures.

The Static Root of Trust

For any attestation to be meaningful, the chain of trust must terminate somewhere in something that is trusted *a priori*—a **root of trust** that is not itself verified by

anything else but is instead assumed trustworthy by construction. The classical approach establishes this root at boot time, yielding a **static root of trust**.

The first mechanism is **secure boot**. Secure boot **verifies system integrity at boot time**, ensuring that only authorized code is permitted to run as the system starts. It relies on an **immutable bootloader stored in ROM along with a public key**: because the bootloader lives in read-only memory, an attacker cannot modify it, and because a trusted public key accompanies it, the bootloader can verify signatures on the code it is about to load. The rule it enforces is simple and strict—it **verifies the code signature and executes the code (that is, boots) only if the signature is valid**. Any unsigned or tampered code is refused, so the very first software to run is guaranteed authentic, and the chain of trust extends upward from there.

The second mechanism is the **Trusted Platform Module** itself, considered now as a root of trust rather than merely a signing engine. The TPM is **present in many commercial systems**, making it a widely available foundation. It contains **Platform Configuration Registers (PCRs)** that are **accessible only via a fixed API**—software cannot write arbitrary values directly into them but may only interact with them through the controlled interface, which is what makes their contents trustworthy. The **PCRs are reset only on boot**, so their values reflect everything measured since the machine started, and they cannot be surreptitiously cleared mid-session. Critically, **each new measurement added to a PCR is "extended" rather than simply stored**: a new measurement is combined with the existing register value by computing a **cryptographic hash of the previous PCR value concatenated with the new measurement**. This "extend" operation gives the PCRs a powerful property. Because each value depends on the entire ordered history of what came before, the final PCR value is a tamper-evident summary of the complete boot sequence—no component can be added, removed, or reordered without producing a different final value, and no later measurement can erase the record of an earlier one.

The Dynamic Root of Trust

The static root of trust measures the system *as it boots*, which means its picture of integrity is fixed at startup and may grow stale as the system runs. Sometimes one needs to establish a trusted measurement of the system's *current* state, long after boot, without rebooting. This is the role of a **dynamic root of trust**.

A dynamic root of trust **can be executed dynamically on the current state of the software**, establishing a fresh, trustworthy measurement at an arbitrary moment during operation rather than only at power-on. This flexibility is valuable precisely because it captures the live state, including any changes that have accumulated since boot. Achieving it, however, **requires specific modules inside the CPU**—special processor instructions and hardware support that can forcibly create a clean, measured execution environment on demand. The representative example is **Intel TXT** (Trusted Execution Technology), which provides CPU-level instructions to launch a measured environment dynamically, resetting the relevant registers and establishing a new root of trust in the middle of a running system.

Limitations of Hardware-Based Attestation

Hardware-based attestation buys stronger guarantees than its software-only counterpart, but it is not free, and its costs define the boundary of where it can realistically be deployed.

The strength is clear: **hardware-based attestation schemes have stronger security properties** than software-based ones, because they do not depend on fragile timing assumptions. But these properties **rely on** specific hardware being present—either a **per-device TPM**, which must be fitted to every single device in the network, or a **trusted BIOS that begins attestation at boot time**, establishing the static root of trust described above. Both prerequisites presuppose that each device carries a nontrivial piece of trusted hardware.

Herein lies the practical limitation. **Employing tamper-resistant hardware on low-end embedded devices is not yet feasible from a cost perspective.** The IoT is populated overwhelmingly by cheap, mass-produced, severely constrained devices—the very perception-layer nodes with which this chapter began—for which adding a full TPM would multiply the unit cost beyond what the application can bear. A full-featured trusted module makes economic sense on a laptop or a server, but not on a one-dollar sensor deployed by the thousands. This tension—strong security requiring expensive hardware that constrained devices cannot afford—is the central dilemma that the next architectural idea sets out to resolve.

Necessary Hardware Components: The SMART Minimal Architecture

The resolution comes from asking a sharper question: rather than demanding a full TPM, what is the *minimum* hardware necessary to make remote attestation secure? This is the question answered by **SMART** (Secure and Minimal Architecture for Establishing a Dynamic Root of Trust), proposed by El Defrawy, Francillon, Perito, and Tsudik. SMART identifies a small set of lightweight hardware features that, together, suffice for secure attestation on even very cheap microcontrollers—far less than a TPM, yet enough to do the job.

Four hardware components are necessary. The first is **Attestation Read-Only Memory**. The attestation code resides inside the microcontroller unit (MCU) in ROM, and crucially, the **key k can only be accessed from this region**. Placing the key behind a hardware rule that ties its accessibility to the ROM-resident attestation code ensures that only the genuine, immutable attestation routine—and no injected malware—can ever read the secret.

The second component is **Secure Key Storage**. This is a dedicated memory region inside the CPU that **can be accessed only from the attestation code in ROM**. It is the physical home of the key, and its access restriction is enforced in hardware, providing the "exclusive access to k" property that the formal model demanded.

The third component is the **MCU Access Controls**, the hardware logic that actually enforces these rules. It **controls access to K and prevents non-attestation code from accessing it**—the circuitry that stands guard, permitting the ROM attestation routine to reach the key while blocking every other instruction stream.

The fourth component is **Reset and Memory Erasure**. This is a fail-safe: **if any error is reported by the above components**—for instance, an attempt by unauthorized code to access the key—**a hardware reset of the MCU is performed, and upon reset, the hardware enforces a memory cleanup**. Any secrets or sensitive intermediate state are wiped, so that a detected violation cannot leave exploitable residue behind.

The memory organization that realizes this can be read off the architecture directly. The MCU core communicates with a memory controller, which in turn governs the device's address space. Within that address space, the **SMART ROM** holds the attestation code, and adjacent to it—inside the protected region—sits the **Key**. Separately lie the general-purpose **SRAM** and **Flash** memories, which hold ordinary, untrusted code and data. The decisive design point is the

placement of the key immediately alongside the SMART ROM and behind the same access-control barrier: the hardware permits the key to be read only when execution is flowing through the SMART ROM code, so untrusted software running from SRAM or Flash is physically incapable of reaching it. With only these few additions to a commodity microcontroller—no full TPM required—SMART provides a hardware-anchored, dynamic root of trust at a cost that even low-end embedded devices can bear.

The SMART Protocol

The behavior of SMART is captured by the interaction between the **verifier (VRF)** and the **prover (PRV)**, together with the algorithm that the prover's SMART ROM executes. At the protocol level the exchange is compact. The verifier **generates a nonce n** and sends to the prover the tuple **(n, a, b, x, x_flag)**. The prover then runs **C = SMART(n, a, b, x, x_flag, −, −)**, and if **x_flag == True**, it additionally performs **Exec(x)**. It returns the result **C** to the verifier, who checks: **if C is correct then Accept, else Reject**.

The parameters carry specific meanings that the underlying algorithm makes clear. The pair **a, b** are the start and end addresses of the memory region to be attested. The value **x** is the address to jump to after attestation, and **x_flag** signals whether such a jump should occur at all—whether the attested code should be executed once verified. The nonce **n** is sent by the verifier to guarantee freshness, and an output address **out** specifies where to store the resulting checksum, with an optional input parameter **in** available for passing data. The output of the whole routine is an **HMAC**.

The ROM algorithm itself weaves together every hardware property discussed above. It **begins by disabling interrupts** during SMART code execution—the hardware realization of the "uninterruptible execution" property, closing the TOCTTOU window by ensuring nothing can interleave with the attestation. The **attestation key K is unlocked automatically by the MCU** precisely because execution has entered the ROM region, after which the routine initializes the HMAC with the key. It then **attests all the parameters** by processing them through the HMAC, binding the token to the exact request (the addresses, the flag, the nonce, and any input). Next it **attests the memory region [a, b]** by iterating over every address i in that range and feeding each memory word into the HMAC, so that the final tag depends on the complete contents of the region under test. The HMAC is then finalized to produce **C**, which is copied to the designated output address.

Before returning control, the routine performs its hygiene and dispatch. It **resets memory to erase temporary variables**, clearing any sensitive residue left by the key-bearing computation. Then it consults the execution flag: **if x_flag is True, it calls the function at address x** (passing the optional input), executing the just-verified code within the still-protected context—so that verification and execution are bound together and no TOCTTOU substitution can intervene. Otherwise, it **restores the interrupt status to what it was before** the SMART execution and returns. In this single, uninterruptible, key-protected routine, SMART delivers the complete chain: exclusive key access, measurement of the exact requested region, freshness via the nonce, and optional trustworthy execution of the attested code—all on hardware barely more complex than a bare microcontroller.

Hybrid Attestation

Between the two poles of pure software attestation—cheap but fragile—and full hardware attestation—robust but expensive—lies a middle path that has become the dominant paradigm for constrained IoT devices: **hybrid attestation**. The hybrid philosophy is to combine a minimal, carefully chosen set of trusted hardware features with a software routine that performs the actual attestation work. The hardware provides exactly the properties that software alone cannot guarantee—exclusive key access, immutability, uninterruptible execution—while the measurement itself remains a software computation. SMART, encountered in the preceding section, is itself an early exemplar of this hybrid approach; the construction described here generalizes the pattern.

The protocol again follows the challenge-response structure, now explicitly partitioning the prover into an **untrusted software** portion and a protected **trusted component**. The prover and verifier **share a key k**. The exchange proceeds in four numbered steps. In **step 1**, the verifier sends a random nonce **N** to the prover. In **step 2**, the trusted component computes **h = HASH(Software)**, a hash over the device's software, reaching across the boundary to measure the untrusted region. In **step 3**, the trusted component computes the attestation token **δ = MAC_k(N ‖ h)**—a message authentication code, keyed with the shared **k**, over the nonce concatenated with the software hash—and transmits δ to the verifier. In **step 4**, the verifier, which expects a particular reference configuration **h′**, computes **δ′ == MAC_k(N ‖ h′)** and tests whether **δ′ = δ**. A match confirms that the device's software hashes to the expected value.

What distinguishes this from pure software attestation is that the hashing and MAC computation occur inside the trusted component, which the hardware guarantees cannot be bypassed, interrupted, or made to leak its key. The security no longer rests on fragile timing; it rests on the hardware's enforcement of the protected region's integrity. The feasibility of this approach is grounded in real hardware that provides exactly such protected regions—among them **TyTAN** and **TrustLite**, as well as commodity technologies such as **ARM TrustZone** and **Intel SGX**—each of which offers a means of isolating trusted code from untrusted code on the same device.

TyTAN: A Tiny Trust Anchor for Tiny Devices

The hybrid model becomes especially interesting when a single embedded device must run **more than one application** at once. A low-end device is no longer imagined as running one monolithic firmware image, but as a small multi-tasking platform hosting several applications of differing sensitivity—some critical, some ordinary, possibly supplied by mutually distrusting parties. **TyTAN** (Tiny Trust Anchor for Tiny Devices), proposed by Brasser, El Mahjoub, Sadeghi, Wachsmann, and Koeberl, is a hybrid architecture designed precisely for this richer setting.

The motivation is economic and architectural at once. Fully **trusted CPUs are expensive**, so TyTAN sets out to design a cheap yet secure architecture for low-end embedded systems that nonetheless provides four substantial guarantees: a **hardware-assisted dynamic root of trust** that allows secure task loading at runtime; **secure inter-process communication (IPC)**; both **local and remote attestation**; and **real-time guarantees**. It is explicitly built for **multi-stakeholder scenarios**, enabling the secure execution of mutually distrusting tasks on the same chip—so that an application from one vendor cannot spy on or corrupt an application from another, even while they share hardware.

The architecture layers a set of services over a modest hardware base. At the hardware level sit the **CPU**, a **platform key (K_P)**, and—the central innovation—an **Execution-Aware Memory Protection Unit (EA-MPU)**. An ordinary MPU enforces which memory regions a given piece of code may access; an *execution-aware* MPU additionally takes into account *which code is currently executing* when deciding whether an access is permitted. This execution-awareness is what allows the hardware to grant a secret, say the platform key, to one specific trusted routine while denying it to all other code, even code that would normally share the same privilege level.

Above this hardware foundation, TyTAN hosts a spectrum of software. It supports ordinary **normal tasks** and isolated **secure tasks**, the latter walled off from all other software including the operating system. Secure tasks provide the security primitives—access control, digital signatures, secure storage—upon which other applications depend, while normal tasks handle non-sensitive work such as a vehicle's collision-prevention logic. A **real-time operating system** schedules these tasks. Notably, this OS is *not* itself a trusted component in the strongest sense; rather, it is an operating system selected to meet the system's requirements and treated as "almost trusted" by virtue of having been deliberately chosen and vetted. The crucial asymmetry is that **secure tasks are isolated from one another, whereas normal tasks are permitted to communicate** freely.

TyTAN's guarantees flow from several interacting mechanisms. Every **task carries a unique identifier**, computed for example as a hash of its own code—so that a task's identity is cryptographically bound to its contents, and any modification changes its identity. The **EA-MPU provides memory access control enforcement**: normal tasks may access only the memory allotted to them, while secure tasks may be granted access to more. To prevent an attacker from jumping into the middle of a task to skip its safety checks—the "controlled invocation" property made concrete—**each task can be invoked only at a dedicated entry point**, so that its first executed instruction is always its true beginning and malware cannot leap back and forth into arbitrary offsets. Interrupts are not forbidden, since a real-time system needs them, but they are carefully constrained: **interrupts are handled such that a malicious interrupt handler cannot gain any information about the state of an interrupted task**, closing a channel through which one task might otherwise spy on another. The **platform key is access-controlled by the EA-MPU**, reachable only by the legitimate trusted code. The real-time OS provides **real-time scheduling**, and **secure boot** ensures that TyTAN's trusted software components are loaded in their correct, authentic versions and isolated from the rest of the system by the EA-MPU from the very first instant—so that an attacker cannot compromise the OS or trusted software before the protections take effect.

The operation of several TyTAN subsystems deserves closer attention. The **EA-MPU driver** sets the memory-access-control rules in the EA-MPU when a secure task is loaded or unloaded, dynamically reconfiguring which regions each running task may touch. The rules for the *static* components—the fixed trusted software present from boot—are established during secure boot and are not altered at

runtime; it is only the rules governing dynamically loaded tasks that change as tasks come and go.

Attestation in TyTAN is performed by having the root of trust compute a cryptographic hash over the binary code of each created task. Because the architecture must honor real-time deadlines, this **attestation must be interruptible during the hash calculation**—a significant departure from the uninterruptible model of SMART. A long hash computation that could not be preempted would risk missing a real-time deadline, so TyTAN permits the hash to be interrupted; it preserves security by ensuring that while a task is being measured, that **task's memory is isolated and its execution is prevented**, so no change can occur between successive fragments of the measurement. The measurement itself is now simple and linear rather than pseudo-random: there is no longer any need to traverse memory in an unpredictable order, because the hardware—not timing—guarantees integrity, so the routine computes a straightforward linear hash. **Local attestation** then compares this hash directly with the task's stored ID, while **remote attestation** additionally involves computing a **MAC** so that the result can be validated by a remote verifier.

Secure inter-process communication addresses the consequence of isolation: because a secure task cannot access another task's memory, two such tasks cannot cooperate except through an explicitly secured channel. TyTAN's IPC therefore includes a **mechanism to authenticate the task sending messages and to limit access to shared memory**. Whoever opens a communication channel—a FIFO or pipe—must be authenticated, so that the receiving task knows exactly which task it is talking to. This authentication is what defeats replay attacks and man-in-the-middle attacks on the inter-task channel, since an impostor cannot masquerade as a trusted sender.

Secure storage builds on these primitives. For each task, a dedicated key is generated as $\mathbf{K_t = HMAC(id_t \parallel K_p)}$—derived by applying an HMAC, keyed with the platform key, to the task's identifier—so that each task's storage key is both unique to that task and unforgeable without the platform key. Tasks interact with a dedicated secure-storage task over secure IPC, and **all data sent to the secure-storage task is encrypted with K_t**, ensuring that data at rest is protected and that one task cannot read another's stored secrets. Overseeing the whole system is a component responsible for **real-time monitoring**, charged with verifying that every real-time application continues to perform its functions without disruption—so that the security machinery does not inadvertently cause a safety-critical task to miss its deadline.

A Vulnerability of Attestation: Denial of Service

Every attestation scheme examined so far shares an implicit structure: the verifier initiates, and the prover must respond. This interactivity harbors a vulnerability. Because the device is obliged to perform a (sometimes costly) attestation computation whenever a verifier asks, an attacker can abuse the mechanism to mount a **denial-of-service (DoS)** attack. The attacker simply **requests an attestation, with a valid-looking nonce, over and over again**, flooding the device with challenges. Each challenge compels the device to drop its real work and grind through an attestation, so that a device under such a barrage is prevented from carrying out its actual function. The security mechanism itself becomes the lever by which the device is disabled.

Non-Interactive Attestation: SeED

The remedy for this DoS vulnerability is to invert the initiative. Instead of the verifier starting the attestation, the **device itself starts the attestation at a moment of its own choosing**—a design known as **non-interactive attestation**. If the device, not an external party, decides when to attest, then flooding it with external requests accomplishes nothing, because those requests no longer trigger any work. This is the approach of **SeED** (Secure non-interactive attestation for Embedded Devices), proposed by Ahmad, Sadeghi, and Zeitouni.

Inverting the initiative, however, creates a new difficulty. If the device self-triggers attestation, a *compromised* device might simply arrange never to trigger, or might detect an impending attestation and clean up its malware just beforehand, then re-infect itself afterward. To foreclose these evasions, SeED augments the minimal SMART hardware with two additional components. The first is a **secure real-time clock (RTC)**: a time source whose value cannot be modified by the attacker. This matters because, if an attacker could learn that an attestation is about to occur and could manipulate the device's sense of time, it could rewind or stall the clock to prevent the attestation from firing; a tamper-proof clock removes this option. The second is an **attestation trigger**: a hardware mechanism that interrupts normal execution and launches the attestation process at the appointed time, together with a **timeout** that governs the scheduling.

The memory and hardware organization reflects these additions. The device retains the SMART hardware features—its ROM-resident attestation code and protected key—and layers on the RTC and the attestation trigger (AT), both fed by the system clock. In RAM reside the operating system, ordinary software, and

a protected block of attestation-related state: the current seed-derived value and the attestation timing variables. In ROM reside the platform key and the secure-boot routine, together with the persistent attestation state. The protected attestation region is placed behind memory-protection rules so that not all code can reach it, and the attacker cannot read the values that determine *when* the next attestation will occur.

The protocol itself is a one-way transmission—the device speaks, the verifier listens—organized around a pseudo-random schedule that both parties can reproduce. The prover maintains a chain of seeds. Given the previous seed value σ_{i-1}, it generates a new pair $\{\sigma_i, \vartheta_i\} \leftarrow gen(\sigma_{i-1})$: a fresh seed σ_i to carry the chain forward, and a value ϑ_i that determines the next attestation interval. From ϑ_i it computes the sleep interval as $t_{att} \leftarrow \vartheta_i \bmod t_{att}^{Max}$, bounding the wait by a maximum, and arms the hardware with $\mathbf{timeout}(t_{att})$. It advances its running record of the attestation schedule, $T_{att} \leftarrow T_{att} + t_{att}$. When the timeout fires, the device takes a software measurement, $h \leftarrow \mathbf{getSoftMeas}()$—the hash of its own software state—and computes the authentication tag $\mu \leftarrow \mathbf{mac}(k_{att}; h \parallel T_{att})$, binding the measurement to the scheduled attestation time. This single message μ is sent to the verifier.

The reproducibility of the schedule is the linchpin. Because both prover and verifier start from the same initial seed σ_0, both generate the identical sequence of seeds, intervals, and attestation times—the verifier can regenerate everything the prover computed and thereby know, independently, both *when* an honest device should have attested and *what* tag it should have produced. On receiving μ, the verifier **regenerates the expected values and checks the tag**. It recomputes the seed chain to obtain ϑ_i and hence t_{att}, advances its own T_{att}, and verifies the MAC with $\mathbf{vermac}(k_{att}; b \parallel T_{att}, \mu)$. If the MAC verifies, it reads the current time $T_{rec} \leftarrow \mathbf{time}()$ and checks that the report arrived within an acceptable window, $T_{att} - \delta_t < T_{rec} < T_{att} + \delta_t + t_{tr}$, where δ_t tolerates clock skew and t_{tr} absorbs transmission delay. Only if the report is both authentic and punctual does the verifier set its result $r \leftarrow 1$, accepting the device. The seed σ lives within the protected memory region, inaccessible to ordinary code, so the attacker cannot predict the schedule.

SeED's resistance to the full adversary hierarchy can now be stated precisely. Against a **communication adversary**, it is secure because the prover and verifier maintain synchronized clocks and reproduce the same schedule; anything out of the ordinary—a missing report, or one arriving at the wrong time—is read as

evidence of a compromised device. Against a **software adversary**, it is secure because the hardware guarantees the integrity of the measurement process and of the clock, and the confidentiality of the key, so malware can neither forge the measurement nor steal the signing key. And against a **mobile adversary**—one able to erase its traces—it is secure because the adversary knows the *previous* attestation time but not the *next* one: the next attestation time is randomized by the pseudo-random generator seeded in protected memory, so the attacker cannot clean up just before a measurement and re-infect just after. For this to hold, neither the triggering code nor any other observable component may reveal the attestation time in advance; the trigger is realized either by polling or, more robustly, as a hardware non-maskable interrupt that the attacker cannot suppress.

The performance profile of SeED is favorable on exactly the axes that matter for constrained devices. It uses **one-way communication**, eliminating the round trip that interactive schemes require. It exhibits **lower runtime** and demands **lower computational power** than its interactive counterparts. When attestation runtime is plotted against software size, three regimes appear: a traditional interactive scheme traces a middle line; a DoS-protected variant of the traditional scheme traces the highest line, paying extra for its protection; and **SeED traces the lowest line**, achieving both DoS resistance and the least runtime of the three. SeED thus delivers DoS protection essentially for free relative to the traditional baseline, because shifting the initiative to the device removes the very channel through which the flooding attack operated.

The Limits of the Single-Prover Setting

Every scheme examined up to this point—software-based, hardware-based, hybrid, interactive, and non-interactive—shares a common horizon: it attests a *single* device. This single-prover framing, adequate for verifying one camera or one sensor, strains badly when confronted with the reality of modern IoT deployments, which routinely involve hundreds, thousands, or millions of devices operating as a collective.

Two limitations make the single-prover setting unsuitable at scale. First, it is simply **not realistic for large-scale implementations**: a verifier that must contact every device individually, one after another, faces a workload that grows linearly with the size of the network, and the aggregate attestation of a large swarm would take prohibitively long. If each device requires its own round-trip exchange and its own verification, a network of a million devices imposes a million such interactions on the verifier. Second, the approach is **time-consuming and**

relies on costly hardware—tamper-resistant modules or TPMs—on the devices, which **makes it unrealistic for large-scale employment**. Fitting expensive trusted hardware to every one of a million cheap nodes, and then attesting each one serially, is economically and temporally infeasible. These twin obstacles motivate a fundamentally different architecture: one that attests an entire network efficiently, rather than one device at a time.

From Device Attestation to Swarm Attestation

To appreciate the shift, it is worth recalling the shape of ordinary **device attestation** in its most distilled form. A single **verifier** issues a **challenge** to a single **prover**; the prover performs its **attestation** computation and returns a **response**; the verifier **verifies the response**. The entire transaction concerns one device and one verifier.

Now imagine this picture multiplied across a dense field of devices—dozens or hundreds of identical embedded boards spread across an environment. The verifier, confronted with this swarm, is overwhelmed: attesting each board individually, through its own challenge-response exchange, does not scale. This is the problem of **swarm attestation**, and it demands that the work of attestation be distributed across the network itself rather than concentrated at a single verifier.

Collective Remote Attestation

The answer is **collective remote attestation (CRA)**. The problem it solves is precisely stated: **to verify the integrity of a network of devices, more efficiently than attesting them individually**. Rather than having a single verifier laboriously poll every prover, CRA enlists the devices to attest one another, so that the verification effort is spread across the whole network and the verifier receives a single, aggregate result.

The architecture involves a population of **provers**—the many devices whose integrity is in question—and a **verifier** that wishes to learn the overall health of the network. The central idea, which recurs across all the CRA protocols that follow, is that each prover does not merely answer the verifier; it also participates in attesting its neighbors, so that **each prover becomes, in effect, the verifier of another device**. This changes the trust landscape in an important way: unlike the single-device setting, one can no longer assume that every verifier is a trusted party, because the "verifiers" are now ordinary devices that may themselves be compromised. A further and decisive benefit of this device-to-device structure is

that **prover-verifier interaction takes place at one-hop distance**—each device attests only its immediate neighbors—which directly solves the multi-hop timing problem that crippled software-based attestation. Because every attestation exchange is local, transmission times remain short and predictable, and the unreliable latency of long multi-hop paths never enters the picture. (The study of CRA at IoT scale, including its state of the art and open challenges, is surveyed by Ambrosin, Conti, Lazzeretti, Rabbani, and Ranise.)

SEDA: Secure Embedded Device Attestation

The first and foundational CRA protocol is **SEDA** (Secure Embedded Device Attestation), proposed by Asokan, Brasser, Ibrahim, Sadeghi, Schunter, Tsudik, and Wachsmann. SEDA organizes the network's attestation around a **spanning tree**: the devices are arranged into a tree rooted at the device nearest the verifier, and the attestation flows through this structure. **Each node attests its children and reports the aggregated result to its parent**, so that results flow upward from the leaves to the root, and the root ultimately delivers a single summary to the verifier.

This design yields two notable advantages. First, SEDA **scales "logarithmically" in the number of provers**: because the tree's depth grows only logarithmically with the number of nodes (for a balanced tree), the time for results to propagate from leaves to root grows far more slowly than the linear cost of attesting each device separately. Second, SEDA **uses symmetric cryptography**, which is cheap enough to run on constrained devices—avoiding the costly public-key operations that would burden low-end hardware.

SEDA's System Model and Assumptions

SEDA rests on two system assumptions. First, **all devices are equipped with a trusted component**—a minimal hybrid-style trust anchor of the kind developed in earlier sections, present on every node so that each can perform a trustworthy local measurement. Second, **devices talk only to their neighbors**: all communication is one-hop, consistent with the locality principle that underpins CRA's resistance to multi-hop timing problems. The verifier sits at the edge of this network, communicating with the root of the spanning tree.

SEDA's Attacker Model

SEDA defends against two classes of adversary. The first is an **eavesdropper**, who can observe the traffic flowing across the network's links. The second is a

software-only attacker—for instance, malware—resident inside a device. Each prover is understood as comprising an **untrusted software** portion, which the attacker may corrupt, and a **trusted component**, which it cannot. Notably, SEDA's threat model does *not* include physical attackers; an adversary who can physically tamper with a device's hardware falls outside the protection SEDA offers, a gap that later protocols set out to close.

The SEDA Protocol

SEDA's operation divides into an **offline** phase, performed once at setup, and an **online** phase, performed at each attestation.

The **offline** phase prepares the devices and their relationships. During **initialization (Init)**, each device is provisioned with an **asymmetric key** and with **certificates attesting its code configuration and public key**. During **registration**, neighboring devices **exchange their certified code configurations** with one another and **establish a shared cryptographic key for communication with each neighbor**. The asymmetric key and certificates serve to guarantee, to the verifier, the authenticity of signatures, while the per-neighbor symmetric keys provide the cheap channels used during the online phase.

The **online** phase is described from the perspective of a single device. A device **receives the global nonce from a neighbor and accepts that neighbor as its parent**—unless it has already received the nonce from someone else, in which case it recognizes that it already has a parent and **returns NULL** to the later requester, so that a device with two potential parents attaches to only one and tells the other to disregard it (this is how a single tree is carved out of a general network). Having accepted a parent, the device **shares the nonce with its own neighbors**, propagating the attestation outward, and **attests itself**. It then **waits for the responses from its children**. Finally, it combines its own result with its children's results and **returns to its parent a signed tuple** $[\mathbf{b}, \beta, \tau]$, together with its configuration. The components of this tuple carry the aggregated verdict: $\mathbf{b}$ is a boolean indicating whether everything in the subtree is fine; β is the count of correctly attested nodes in the subtree; and τ is the total number of nodes in the subtree. Optionally, the response may also carry the **IDs of any compromised devices**, so that the verifier learns not merely *how many* nodes failed but *which* ones. As these tuples flow upward and are combined at each level, the root eventually holds a complete tally for the whole network, which it reports to the verifier.

SEDA's Limitations

For all its pioneering value, SEDA carries three significant limitations, each of which drives the development of its successors.

The first is a **lack of flexibility**: in SEDA, *all* devices must participate in the attestation. There is no provision for attesting only a subset of the network, nor for gracefully handling devices that are temporarily unavailable; the protocol presumes a fixed, fully participating population.

The second is that the **final attestation result is not publicly verifiable**. Because SEDA relies on symmetric cryptography and shared keys, the aggregated verdict can be checked only by a party holding the appropriate secrets. A third party cannot independently confirm the result, which limits the protocol's usefulness in settings where the outcome must be auditable by others.

The third, and most serious, is that SEDA **fails against physical attacks**. This weakness is intrinsic to the tree structure. Consider an attacker who physically compromises an *internal* node of the spanning tree—a node with children of its own. Because that node is responsible for collecting, combining, and forwarding the attestation results of its entire subtree, a physically compromised node **can arbitrarily modify the attestation results of its subtree**. It might, for instance, receive honest reports from its children indicating that several of them are compromised, and simply rewrite the aggregate to claim that all is well before passing it upward. Since the parent trusts the tuple it receives, the deception propagates all the way to the verifier. The tree's efficiency—having each node summarize its subtree—is thus also its Achilles' heel, because a single corrupted intermediary can falsify the verdict for everything beneath it.

SEDA's Performance

SEDA's experimental evaluation confirms its favorable scaling and characterizes its costs along three dimensions.

Regarding overall **performance for tree topologies**, the run-time is plotted against the number of devices in the swarm for trees of varying branching factor—binary, 4-ary, 8-ary, and 12-ary. All of these curves rise steeply at first and then flatten into a plateau, so that run-time grows only very slowly as the swarm expands from tens of thousands to a million devices. This flattening is the visible signature of logarithmic scaling: beyond a certain point, adding more devices barely increases the total attestation time. Among the branching factors, the

trade-off is that higher branching reduces tree depth (fewer levels to traverse) but increases the work at each node (more children to attest), with the 4-ary tree offering a favorable balance.

Regarding **run-time per device**, the cost is plotted against the number of neighbors a device has, separately for an initiating device and for other devices, under two different trusted-hardware bases—SMART and TrustLite. The run-time of the device that *initiates* the attestation, using the heavier SMART base, rises most steeply with the number of neighbors, reaching several seconds; the other devices under SMART rise less steeply; and both the initiator and other devices under **TrustLite remain very low and nearly flat**, demonstrating that the choice of trust anchor strongly influences per-device cost and that a well-chosen anchor keeps it modest even as connectivity grows.

Regarding **energy consumption**, the cost is plotted against the number of neighbors for two sensor-node platforms, MICAz and TelosB, again distinguishing the initiator from other devices. The initiator on the MICAz platform consumes the most energy, rising from roughly 50 millijoules upward as neighbors increase; the initiator on TelosB consumes substantially less; and the *other* (non-initiating) devices on both platforms consume very little, their curves hugging the bottom of the chart. The pattern confirms that the bulk of the energy burden falls on the initiating node, while ordinary participants pay only a small energy price—an encouraging property for battery-powered swarms, provided the initiator role is not too onerous.

SANA: Secure Aggregate Network Attestation

SEDA established that a network could be attested collectively through a spanning tree, but it left three wounds open: a compromised internal node could falsify its subtree's verdict, the final result could not be publicly verified, and the scheme buckled against physical attackers. **SANA** (Secure Aggregate Network Attestation), proposed by Ambrosin, Conti, Ibrahim, Neven, Sadeghi, and Schunter, is engineered to heal exactly these wounds. Its central technical device is an **aggregate signature scheme** that allows many devices' signed proofs to be combined into one compact, publicly verifiable signature, with security that survives even the physical compromise of the intermediaries doing the combining.

SANA works over a network of **heterogeneous devices**, each of which **talks only to its neighbors** and which together can **form an overlay aggregation tree**. What distinguishes SANA's architecture is that it recognizes four distinct

roles: **provers**, **aggregators**, an **owner**, and a **verifier**—and, crucially, a single device may play more than one role, so that **a device may be both a prover and an aggregator** simultaneously. In the topology, leaf and interior devices act as provers (labeled P_1 through P_5), certain interior devices additionally serve as aggregators (labeled A_1 and A_2) that combine the proofs flowing through them, the **owner** holds the reference information defining what a healthy device looks like, and the **verifier** ultimately receives and checks the aggregated result.

SANA's Attacker Model

SANA's threat model is strictly stronger than SEDA's. It defends against the familiar **eavesdropper** and **software-only attacker**, but it now adds a **physical attacker** to the roster. This is the decisive advance. The way SANA accommodates physical compromise is subtle and rests on separating the trust requirements of the two roles. A physically compromised **aggregator** is tolerated, because—as will be seen—it cannot forge a valid attestation result even if it is fully in the attacker's hands. The remaining limitation is that a physically compromised **prover** can evade detection, since an attacker who has physically extracted a prover's secrets can produce a valid-looking proof for that one device; but it cannot thereby corrupt the verdicts of *other* devices. The attacker is modeled as able to sit on the links (eavesdropping), inside devices' untrusted software, and—for aggregators and provers alike—physically present, while the owner and verifier remain trustworthy anchors.

SANA's Core Idea

The elegance of SANA lies in how it distributes trust requirements asymmetrically between provers and aggregators, so that the expensive, highly-trusted hardware is needed only where it is unavoidable.

Only the **provers (P_j)** require a **Trusted Execution Environment (TEE)**. Each prover, from within its TEE, **builds an attestation proof**. This proof **contains a digital signature σ_j over the measured configuration**, and—because SANA aims at public verifiability—**σ_j is designed to be publicly verifiable**: anyone holding the appropriate public key can check it, without needing any shared secret. The use of digital signatures rather than MACs is precisely what delivers the public-verifiability property that SEDA lacked.

The **aggregators (A_i)** have a different job: they **"combine" the proofs of the provers**. For this combination to be efficient, the signatures must be **aggregatable**—meaning that many individual signatures can be merged into a

single short signature whose size does not grow with the number of signers. This is the technical heart of SANA: an aggregate-signature construction lets an aggregator fold together the proofs of its entire subtree into one compact object, so that the final result transmitted to the verifier remains small regardless of how many devices contributed.

The security payoff of this division is profound and directly addresses SEDA's fatal flaw. Because the individual signatures are generated inside the provers' TEEs and merely combined (not fabricated) by the aggregators, a **physically compromised aggregator cannot forge the attestation result**. An aggregator can combine the signatures it receives, but it cannot invent a valid signature for a configuration that no honest prover actually signed—so it **cannot mount a man-in-the-middle attack** by rewriting its subtree's verdict. This is exactly the attack that broke SEDA, and SANA neutralizes it by ensuring that the authority to vouch for a device's state rests solely with that device's TEE-protected signature, not with the intermediary that forwards it. The residual weakness is narrow and acknowledged: a **physically compromised prover can evade detection** for itself, but the damage is confined to that single device and cannot spread to falsify the network's overall result.

Network Software Integrity Verification: The SANA Flow

The full SANA procedure can be traced as a flow of information down the tree from the verifier and back up through the aggregators, culminating in a single verification at the verifier.

The challenge descends. The verifier constructs a challenge of the form $\mathbf{Ch = \{N, T = \{h_g, H\}\}}$ and disseminates it down through the aggregation tree—from the verifier to A_2, from A_2 to A_1 and to the other provers it parents, from A_1 to its own children, and so on until every device has received **Ch**. Within this challenge, **N** is the familiar random nonce ensuring freshness. The set **H** is the set of **"known good" configurations**—the reference list, supplied by the owner, of every software state a device is permitted to be in. And $\mathbf{h}g = HASH(h_1 \,\|\, \ldots \,\|\, h\{|\mathbf{H}|\})$ is a single hash computed over the concatenation of all the known-good configuration hashes—a compact digest of the entire set of acceptable states.

Each prover attests and produces a proof. Every device measures its own configuration and compares it against the set of known-good configurations encoded in the challenge. The outcome determines the content of the proof it

generates. If the device's **configuration is "good"—that is, present in H**—then the resulting proof contains an **aggregatable (OAS) signature over h_g ‖ N ‖ ...**: by signing the digest of the good set together with the nonce, the device certifies "I am in one of the approved states, freshly, for this challenge," without needing to reveal which specific good state it is in. If, on the other hand, the configuration is **not good**, the proof instead contains an **OAS signature over the "bad" configuration** actually measured, together with the **device's public key pk_i**—so that the verifier will learn both that this device is compromised and precisely which device and which bad configuration are at fault.

Aggregators combine proofs as they ascend. As the proofs flow back up the tree, each aggregator **aggregates the signatures it receives and propagates the public keys of any provers reporting a "bad" configuration**. The crucial efficiency is that the many "good" signatures—all over the same h_g ‖ N—collapse into a single aggregate signature, while only the comparatively few "bad" reports need to be carried individually along with their identifying public keys. At aggregator A_1, the good proofs from its subtree are merged into one; at A_2, A_1's aggregate is further merged with A_2's own subtree, and the equality of the combined good-signatures confirms they all signed the identical h_g ‖ N. Step by step, the whole network's good-device attestations condense into one signature, accompanied by a short list of the bad devices' identities.

The verifier checks once. Ultimately a single aggregated proof reaches the verifier, who performs **Verify(·)** on it. Because the good signatures have all been aggregated into one, the verifier confirms in a single check that every device whose signature was folded in was indeed in a good state, and it reads off, from the propagated public keys, exactly which devices were bad. The computational cost of this final verification is **linear in the number of bad provers**, which in a healthy network is far smaller than N, the total number of devices. This is a remarkable property: the verifier's work scales not with the size of the network but with the (hopefully tiny) number of compromised devices, so that attesting a vast but mostly-healthy network costs almost nothing at the verifier.

SANA's Improvements

Gathering these threads, SANA's contribution over the prior state of the art can be stated crisply. It **allows end-to-end secure and scalable attestation**, and it improves upon earlier protocols in three specific ways. It **guarantees public verifiability**, so that the attestation result can be checked by any party rather than only by a holder of shared secrets. It **considers less strict requirements**

for protocol entities, demanding a full TEE only of provers and tolerating weaker assumptions elsewhere. And it **provides better resilience to both physical attacks**—since compromised aggregators cannot forge results—**and denial-of-service attacks**.

SANA's Performance

SANA's run-time is evaluated against the number of devices in the network, holding fixed a certain number of bad provers, considering fifty different bad configurations, and assuming that aggregators are more powerful devices than ordinary provers. The comparison pits SANA against SEDA and against an identity-augmented variant called SEDA-ID (SEDA extended to report the identities of compromised nodes). Plotted across networks scaling up to a million devices, the curves for 4-ary trees reveal that SANA's run-time rises to a modest plateau and then stays essentially flat as the network grows, remaining close to that of SEDA-ID for large networks. The headline result is that SANA achieves **comparable run-time to SEDA-ID for large networks** while delivering its far stronger security guarantees—public verifiability and resilience to physical attacks—essentially without a scalability penalty. SEDA itself, lacking identity reporting, runs fastest but offers the weakest guarantees; SANA buys dramatically better security for a cost that, at scale, is negligible relative to the identity-aware baseline.

DARPA: Device Attestation Resilient to Physical Attacks

SANA tolerated physical attackers by ensuring that a compromised aggregator could not *forge* a result, but it did not actively *detect* when a device had been physically tampered with. **DARPA** (Device Attestation Resilient to Physical Attacks), proposed by Ibrahim, Sadeghi, Tsudik, and Zeitouni, attacks the problem from a complementary angle. Rather than only verifying software integrity at attestation time, DARPA adds a mechanism for **continuously monitoring the physical presence of devices in the intervals between attestations**, so that the act of physically removing a device—which any hardware tampering requires—can itself be caught.

The insight behind DARPA concerns the physical mechanics of a hardware attack. To physically compromise a device, an attacker must take it offline: a device has to be physically removed from the network, have a component swapped or modified, and then be reinserted. This removal necessarily creates a window during which

the device is absent from the network. DARPA is designed to notice that absence. By doing so, it closes a variant of the time-of-check-to-time-of-use gap at the hardware level—an attacker cannot quietly extract a device, tamper with it, and slip it back without the gap in its participation being observed by its neighbors.

The mechanism works through a protocol of periodic liveness announcements. In the time between two successive attestations, **each device in the network periodically monitors the other devices**. Every device **broadcasts its heartbeat—an announcement of its continued presence, or "attendance"—to its neighbors**. Each device **collects the heartbeats it receives from its neighbors and broadcasts them onward to its own neighbors, and so on**, so that knowledge of each device's liveness propagates across the network. These **heartbeats are stored into logs by every device**, building up a distributed record of which devices have been continuously present and which have gone missing. When the time comes to attest, the network **performs SEDA to collect the heartbeat logs**, aggregating this presence information up the spanning tree in the same manner that SEDA aggregates attestation results. A device that disappeared from the heartbeat record during the inter-attestation interval—because it was physically removed for tampering—is thereby flagged, even if, after reinsertion, its software appears to attest correctly.

The trade-offs of this design are clear. On the positive side, DARPA is **able to address physical adversaries along with the software and hybrid adversaries** handled by earlier schemes: the heartbeat mechanism catches the physical-removal signature that pure attestation misses. On the negative side, DARPA **inherits the same flaws as SEDA**, upon which it is built. It is **not suitable for dynamic networks**, because the spanning-tree structure and the heartbeat neighborhoods assume a relatively stable topology; and the **details of compromised devices are not present in the attestation report**, since the SEDA aggregation it relies upon does not natively carry per-device identity information. There is, moreover, an implicit cost worth naming: the continuous broadcasting and relaying of heartbeat messages consumes power, which makes such a solution poorly suited to low-battery devices for which every transmitted message is an expenditure of a scarce energy budget.

A Comparative View: Other Spanning-Tree Protocols

DARPA, SEDA, and SANA are the most prominent members of a larger family of CRA protocols all built upon the spanning-tree idea. Three further protocols

round out this landscape, each improving on its predecessors along a particular axis while sharing the family's characteristic weakness in the face of highly dynamic networks.

Lightweight Swarm Attestation (LISA), presented by Carpent and colleagues, improves on SEDA in terms of **scalability and resiliency**. Its distinctive contribution is conceptual as much as technical: it introduces a metric called **Quality of Swarm Attestation (QoSA)**, a yardstick for comparing the quality of different attestation schemes along dimensions such as how much information the final report conveys. Its limitations are that it **requires full connectivity during attestation**, that it **does not support device mobility during attestation**, and that, like its relatives, it is **unsuitable for dynamic networks**.

SCAPI (A Scalable Attestation Protocol to Detect Software and Physical Attacks), by Kohnhäuser and colleagues, improves on DARPA in terms of **resiliency**, and it likewise **mitigates denial-of-service attacks and addresses physical adversaries**. Its limitation is that it is **not suitable over highly dynamic networks**.

SeED (Secure Non-Interactive Attestation for Embedded Devices), by Ibrahim and colleagues—encountered earlier as the non-interactive single-device scheme—also belongs to this comparative picture at the swarm level. Its contribution is the introduction of a **non-interactive attestation scheme**, and it too **mitigates denial-of-service attacks and addresses physical adversaries**. Its limitation, again, is that it is **not suitable over highly dynamic networks**.

The recurring refrain across this entire comparison is unmistakable: every spanning-tree protocol, however it improves on scalability, resiliency, DoS resistance, or physical-attack detection, stumbles when the network becomes **highly dynamic**—when devices move, join, leave, and change their neighbors during the attestation itself. The spanning tree is a static scaffold, and a static scaffold cannot be erected on shifting ground. This shared limitation is precisely the frontier that the final family of protocols, built for mobile and disruptive networks, sets out to conquer.

From Static Trees to Dynamic Swarms

The spanning-tree protocols share a hidden assumption that their recurring failure mode lays bare: they presume that the network's topology holds still long enough

for a tree to be built and used. In a **tree-based collective remote attestation**, the devices are arranged into a fixed hierarchy—a verifier at the root, and beneath it a cascade of parent-child links through which challenges flow downward and aggregated results flow upward. Each link is a committed relationship: a parent attests specific children, and those children report to that specific parent. The whole edifice depends on these relationships persisting for the duration of the attestation.

But many of the most compelling IoT scenarios involve devices that will not hold still. Picture a swarm of drones in flight, or a fleet of mobile robots, or any collection of nodes whose physical positions—and therefore whose radio neighborhoods—change continuously. In such a setting, the parent-child links of a spanning tree dissolve as fast as they are formed: a node that was a child of one device a moment ago may have flown out of range and into the neighborhood of an entirely different device. A drone that is attesting a neighbor may, mid-protocol, lose contact with that neighbor and gain three others. This is **dynamic collective remote attestation**, and it demands a fundamentally different organizing principle. One cannot build a rigid tree on a swarm whose edges are in constant flux; the attestation must instead be structured so that it tolerates—indeed, embraces—continual change in who is talking to whom.

PADS: Practical Attestation for Highly Dynamic Swarm Topologies

The protocol that breaks decisively with the tree is **PADS** (Practical Attestation for Highly Dynamic Swarm Topologies), proposed by Ambrosin, Conti, Lazzeretti, Rabbani, and Ranise. PADS abandons the spanning tree altogether and replaces it with a principle borrowed from distributed computing: **consensus**, achieved through the continual, opportunistic exchange of information among whatever neighbors happen to be in range at any moment.

PADS operates over **heterogeneous devices**, each of which **talks only to its neighbors**, but with a defining new characteristic: the **devices are mobile and do not form an overlay spanning tree for attestation**. Several design commitments follow from this. Attestation is based on **self-attestation**: rather than one device attesting another, every device measures *itself*, and all devices begin their local self-attestation together, synchronized by a real-time clock and an attestation trigger (the same non-interactive machinery developed for SeED). Having each attested itself, the devices then use the concept of **"consensus" among themselves to corroborate the attestation result**—they spread their

individual findings across the swarm so that, through repeated exchange, knowledge of the whole network's state diffuses to every participant. A powerful consequence is that the **verifier can obtain a "snapshot" of the network at runtime**: at any moment, because the devices have been continually sharing, each device holds an approximate picture of the entire network's health, and the verifier can read off that picture by attesting any single device and querying its accumulated knowledge. This diffusion-by-repeated-local-exchange is the classic distributed-systems pattern known as **gossip**: information propagates the way a rumor spreads through a crowd, hop by hop, until everyone has heard it.

PADS Device Requirements

PADS asks relatively little of each device's hardware, keeping to the minimal hybrid foundation established earlier. Each device needs a **Read-Only Memory (ROM)**, a **Memory Protection Unit (MPU)**, and a **secure Real-Time Clock (RTC)**. The role of each is visible in the device's memory organization. The CPU is driven by a clock system that supplies both the RTC and an attestation trigger (AT). The MPU governs access to memory according to a table of rules, each rule naming an allowed code region, a memory region, and an access policy (read, or read-and-write). The protected regions of memory hold the attestation-critical state: in one region reside the attestation variable **v_att**, the set of known-good configurations **H = {h₁, ..., h_M}**, and the attestation key **k_att**; another region holds the **PADS routines** and the operating system. The ROM holds the **platform key k_platform** and the **secureBoot()** routine. The MPU's rules are arranged so that the PADS routines may read the sensitive attestation state while ordinary software cannot, enforcing—on cheap hardware—the confidentiality that the consensus mechanism depends upon.

PADS Entities and the State Encoding

As in SANA, **only the provers (P_j) require a Trusted Execution Environment (TEE)**, and each prover **builds an attestation proof** from within it. This proof **contains the hash value of the underlying software**, and it encodes a verdict drawn from **three possible states**, represented by two bits each: **Good (10)**, **Bad (00)**, and **Unknown (11)**. The three-state encoding is essential to a dynamic setting, because a device may genuinely not yet know the status of some far-off node it has not heard about—"unknown" is a first-class answer, distinct from both "healthy" and "compromised." Every prover **shares its knowledge with the other nodes in range**, which is the mechanism by which the gossip proceeds. The **verifier (vrf)** participates by first attesting an

individual node before trusting and reading its accumulated knowledge about the network. As with SANA, the residual weakness is that a **physically compromised prover can evade detection** for itself, though it cannot corrupt the collective picture.

The representation that makes this scalable is a compact binary string. To attest a network of **N devices**, PADS uses a binary string of **2N bits**—two bits per device, matching the three-state encoding. Each device is assigned a device number **i**, and the two bits recording that device's state occupy positions **2i and 2i+1** of the string. In this way, the complete status of the entire network—good, bad, or unknown for every one of its members—is packed into a single bitmask of length 2N that every device carries, updates, and shares. When a device learns something new about node i, it writes the corresponding verdict into those two bit positions.

Maximum Consensus

The rule by which devices reconcile their differing bitmasks is called **maximum consensus**, and it is designed for efficient network-status communication among devices. When two devices meet and exchange their observations, each updates its own bitmask by taking, position by position, the "maximum" of its current value and the values reported by the neighbors currently in range. Formally, a prover's observation at the next step is computed as

$$\mathbf{x}i\hat{}\{t+1\} = maxatt(\ x_i\hat{}t,\ \{x_j\hat{}t\}\{\mathbf{P}j \in N\{\mathbf{i,t}\} \cap \mathbf{R_t}\}\ ),$$

where the update is applied componentwise: for each position l,

$$\mathbf{x}i\hat{}\{t+1\}[l] = max(\ x_i\hat{}t[l],\ \{x_j\hat{}t[l]\}\{\mathbf{P}j \in N\{\mathbf{i,t}\} \cap \mathbf{R_t}\}\ ),$$ for l = 0, …, m.

Here **x_i^t** is the network observation held by prover i at step t; **m** is the length of the Bloom filter (introduced in the next part of the course); **N_{i,t}** is the set of neighbors of prover P_i at step t; and **R_t** is the set of active and responsive provers at step t. The device thus merges its own view with the views of exactly those neighbors that are both in range and responsive at that moment—which is precisely the set that shifts continually in a mobile swarm.

The semantics of taking the maximum encode a deliberate security policy. The three states are ordered by their bit patterns so that taking the maximum resolves disagreements conservatively. Concretely, if two devices hold **discordant**

opinions about a given node—one asserting that, say, device 1 is compromised and another asserting it is not—then the consensus rule resolves the conflict by **declaring the device compromised**. A "bad" or suspicious verdict, once it appears anywhere in the swarm, propagates and dominates, so that a compromise detected by even a single honest observer cannot be erased by others' more optimistic reports. The operation is, in effect, **equivalent to a bitwise OR** across the devices' observations, which monotonically accumulates evidence of compromise and never discards it. This monotonicity is what allows the gossip to converge: evidence only ever grows, so the shared picture stabilizes as information spreads.

The PADS Protocol in Motion

The life of a PADS attestation can be traced through four phases, which together show how a global verdict crystallizes out of purely local exchanges.

Initialization. Every device begins with its bitmask set entirely to zeros—a blank slate in which it has recorded nothing about anyone, including itself. The verifier stands by, ready eventually to query the swarm. At this moment no device knows anything about the network's state.

Self-attestation. Triggered together by their synchronized clocks, every device measures its own software and writes its own verdict into its bitmask. An honest device records itself as "healthy" in its own two bit positions, while a device that detects corruption records "compromised." After this phase, each device knows its *own* status with certainty but still knows nothing about the others—the bitmasks differ from device to device, each populated only in the slot corresponding to itself.

Consensus. The devices now begin to gossip. Whenever two devices come within range—a relationship that changes constantly as the swarm moves—they exchange their MAC-protected observations for the current time step and each applies the maximum-consensus (bitwise-OR) update. Through repeated such encounters, the verdicts recorded by individual devices spread outward: a "compromised" mark placed by one device on itself, or by one observer on a neighbor, diffuses across the swarm until many devices carry it. Over successive rounds the bitmasks grow richer and begin to agree, converging toward a common picture of the whole network.

Verification. Eventually the verifier obtains the **network state** by reading the accumulated bitmask from a device it has attested. A device that has been in contact with much of the swarm will hold a nearly complete picture and can

report the overall state. But the dynamic setting introduces two honest caveats that the verifier must respect. A device that has had limited contact may have **no complete knowledge**—its bitmask still contains "unknown" entries for nodes it never heard about—and a device that has drifted **out of the communication range** of the rest of the swarm simply cannot have gathered or contributed recent information. The three-state encoding is what makes these situations expressible: "unknown" honestly signals the gaps that mobility inevitably creates, rather than forcing a device to guess.

Interpreting the Result

Once the verifier holds a device's bitmask, it can extract two kinds of conclusion. It can **estimate the number of compromised devices** by counting the "bad" entries across the mask, obtaining an overall measure of the network's health. And it can **determine whether a specific device is compromised**, reading off that device's two bit positions—though, in the dynamic and gossip-based setting, this determination carries some **uncertainty**, since the information may be incomplete or stale depending on how thoroughly it has propagated. The verifier thus gains a probabilistic, network-wide snapshot rather than the crisp, exhaustive verdict that a static tree could in principle provide—a reasonable price for the ability to attest a swarm that never stops moving.

PADS Security

PADS is **secure against communication adversaries, software adversaries, and mobile software adversaries**—the three software-level threat classes developed at the outset of the chapter. Against these, the combination of self-attestation, MAC-protected exchanges, protected memory, and the monotonic consensus rule holds. The natural question is what an adversary *can* still do within this model, and the honest answer is narrow: the principal residual action available to an adversary is **not participating in the protocol**—simply staying silent, refusing to broadcast or relay. But silence is itself informative in a consensus scheme, because a device that never contributes its "good" status, and about which neighbors can learn nothing, will show up in others' bitmasks as "unknown" rather than "good," which already signals that something may be amiss.

PADS Performance

PADS is evaluated by its **run-time in seconds for different coverage levels**—specifically, the time for 85%, 90%, or 95% of the total devices in the network to

acquire knowledge (coverage) of 95% of the network. Plotted against the number of devices, the run-time curves for all three coverage targets rise together: nearly flat and low for small networks, then climbing increasingly steeply as the network grows into the thousands, reaching on the order of 40–48 seconds for a swarm of roughly 8,000 devices. The three curves sit close to one another, with the most demanding target (95% of devices reaching 95% coverage) highest, as expected. The upward bend at scale reflects the cost of diffusing information across an ever-larger mobile population through purely local exchanges.

PADS Pros and Cons

PADS's improvements over its tree-based predecessors are exactly the capabilities the dynamic setting demanded. It is **suitable for dynamic networks**; it **considers device movement during attestation** rather than assuming a frozen topology; and it allows the **verifier to obtain a snapshot of the network at run-time** through the gossip-maintained shared state.

Its drawbacks, however, stem from the rigidity of its membership model. In PADS, **nodes cannot easily join the network**, because the fixed-length 2N-bit bitmask and the per-device bit assignments are baked in from the start. Consequently, **adding a new node requires reconfiguration of all the devices**: every device's bitmask must be resized and its indexing updated to accommodate the newcomer, a global reconfiguration that is awkward precisely in the open, fluid environments where new devices might naturally wish to appear. This tension—between embracing *mobility* while still struggling with *membership change*—is the gap that the final protocols of the course, built on Bloom filters, set out to close.

SALAD

Before arriving at the Bloom-filter construction that resolves PADS's membership rigidity, one intermediate protocol deserves mention. **SALAD** (Secure And Lightweight Attestation of highly Dynamic and disruptive networks), proposed by Kohnhäuser, Büscher, and Katzenbeisser, targets the same regime as PADS—highly dynamic and disruptive networks—but takes a different approach to disseminating attestation information.

SALAD's distinguishing features, and their trade-offs, can be summarized directly. On the positive side, it uses a **broadcast of challenges and responses**, spreading attestation information through the network by broadcasting rather than through structured point-to-point exchanges; it **provides a report for each**

device, yielding per-device verdicts rather than only an aggregate count; and it rests on **strong cryptographic assumptions**, giving it robust security guarantees. The cost of these properties is a **high overhead**: the broadcasting of challenges and responses, combined with the strong cryptography, makes SALAD comparatively expensive to run.

Internally, SALAD employs **three different types of aggregation**, named **MACGreedy**, **MACSimple**, and **MACSmart**, which differ in how they combine the MAC-protected reports as they propagate. Their relative performance is visible when one plots, for a network of 3,000 devices under high dynamics (with device speeds of 10–20 meters per second), the fraction of the final report that an average device has stored as time passes. For a given MAC-tag length, **MACSmart** causes devices to acquire the complete report fastest, **MACGreedy** is slowest, and **MACSimple** lies between them; the average time until the first and last network devices store the final report is marked by vertical lines on such plots. Comparing across tag lengths, shorter tags (for instance 128-bit versus 256-bit) let the report propagate faster because less data must be moved, and increasing the number of MAC computations per message likewise shifts the completion times. SALAD thus offers a family of aggregation strategies trading speed against the robustness of its guarantees, but always at a notably higher overhead than the lighter consensus approach of PADS.

Bloom-Filter-Based Collective Remote Attestation

The final and most refined protocol in this progression is the **Bloom-filter-based collective remote attestation** scheme of Frontera and Lazzeretti. It inherits PADS's gossip-and-consensus philosophy for dynamic swarms but replaces the rigid, fixed-length 2N-bit state representation with a **Bloom filter**, a probabilistic data structure that dissolves exactly the membership-rigidity problem that burdened PADS. By doing so it retains PADS's advantages while permitting devices to join and leave fluidly.

The Bloom Filter

A **Bloom filter** is a **space-efficient probabilistic data structure** used to test whether an element belongs to a set. Its purpose here is **element verification in the set**—answering the question "is this item a member?"—while occupying far less space than storing the set explicitly. Its operation **requires multiple hash functions**. To insert an element, each of the k hash functions is applied to it, and

each produces an index into a bit array; the bits at all those indices are set to 1. To test membership, the same hash functions are applied to the query element, and if *all* the corresponding bits are set, the element is reported as present.

The structure's defining characteristic—and the source of the word "probabilistic"—is that it can produce **false positives but never false negatives**. This is illustrated by considering several elements against a filter. Elements **x, y, z** have genuinely been inserted, so each maps, through the several hash functions, to bit positions that have all been set to 1; a membership test for any of them succeeds correctly. An element **w** that is **not in the set** maps to at least one position that remains 0, so the filter correctly reports it absent. But an element **u** may be a **false-positive element**: although u was never inserted, every one of its hashed positions happens to have been set to 1 by the insertions of *other* elements, so the filter mistakenly reports u as present. False negatives, by contrast, are impossible—an element that was truly inserted can never test as absent, because its bits, once set, are never cleared.

This asymmetry is precisely what makes the Bloom filter suitable for attestation consensus. The filter can be used, for example, to determine whether a particular secret key has been used, or—in the swarm setting—to record which devices have been observed in a particular state. Because the filter only ever accumulates set bits and never clears them, it is naturally monotonic, matching the bitwise-OR consensus semantics that PADS relied upon.

The filter also supports an estimate of how much it contains. Given a filter of **m** bits using **k** hash functions, of which **X** bits are currently set, the approximate number of elements inserted is

$$\mathbf{z = -(m/k) \cdot \ln(1 - X/m)}.$$

This formula lets a device estimate how many items have been folded into a filter purely from the count of set bits, without enumerating the elements—a capability that, in the attestation setting, allows the verifier to estimate how many devices have contributed to the consensus.

Self-Attestation with a Trusted Challenge

As in the non-interactive and dynamic schemes before it, the Bloom-filter protocol avoids the DoS vulnerability of verifier-initiated attestation by having **the challenge generated by a trusted component inside the device itself**. The hardware requirements mirror those of PADS: a **TrustLite-style platform**

augmented with a real-time clock, with the attestation **parameters stored in writable memory** but **read-write protected by the MPU**, and a **secure boot** that guarantees the integrity of the attestation code (here the BADS routines). The memory layout parallels PADS's: the CPU is clocked by a system that drives both the RTC and an attestation trigger (AT); the MPU enforces a rule table of allowed-code-region, memory-region, and access-policy triples; the protected RAM region holds the attestation variable v_att, the known-good set H = {h_1, …, h_M}, and the attestation key k_att, alongside the attestation routines and the OS; and the ROM holds the platform key and the secure-boot routine. The device therefore self-triggers its attestation on a tamper-proof schedule, closing the DoS channel while keeping the attestation state confidential.

Maximum Consensus over Bloom Filters

The consensus rule is identical in form to PADS's maximum consensus, now understood as operating over the Bloom-filter bit array. Each prover updates its filter by taking the componentwise maximum of its own filter and those of the in-range, responsive neighbors—**x_i^{t+1}[l] = max(x_i^t[l], {x_j^t[l]})** for each of the m bit positions—which, over binary values, is once again **equivalent to a bitwise OR**. The parameters carry the same meaning as before: x*i^t is prover i's observation at step t, m is the Bloom filter length, N*{i,t} is prover i's neighbor set at step t, and R_t is the set of active and responsive provers. When two devices meet, their filters are OR-ed together, so that any device recorded in either filter becomes recorded in both—evidence accumulates and never vanishes, exactly as the monotonic Bloom filter requires.

The Protocol in Motion

The dynamics mirror PADS's four phases, now with each device carrying a Bloom filter rather than a fixed 2N-bit mask.

Initialization. Every device starts with its Bloom filter cleared to all zeros, having recorded nothing about the network.

Self-attestation. Triggered together by their synchronized real-time clocks, the devices each measure their own software and declare a verdict—"healthy" or "compromised." An honest device inserts the appropriate token into its own filter; a device detecting corruption records the compromised verdict. In the illustrative snapshot, most devices announce "healthy" and set the corresponding bits, while a few that detect tampering announce "compromised" and set bits encoding that fact

—so that the individual filters begin to differ, each reflecting its owner's self-knowledge.

Consensus. The devices gossip. As mobility brings pairs of devices into range, they exchange their filters and apply the bitwise-OR update, and the recorded verdicts—healthy and compromised alike—diffuse across the swarm. A "compromised" mark, once inserted anywhere, propagates and can never be cleared, so the shared picture monotonically accumulates the evidence of which devices are in which state.

Verification. Eventually the verifier reads the accumulated filter from an attested device to recover the **network state**. From that filter it can, using the element-count formula, estimate how many devices have been recorded, and it can test individual devices for membership in the "compromised" set—subject to the Bloom filter's intrinsic false-positive possibility.

Security

The Bloom-filter scheme is **secure against communication adversaries, software adversaries, and mobile software adversaries**, the same three software-level threat classes that PADS withstands. The mechanisms are the same: self-attestation anchored in protected hardware, MAC-protected exchanges, confidential attestation state behind the MPU, and a monotonic consensus rule that evidence of compromise can only enter, never leave. As in PADS, the residual action available to an adversary is essentially to **decline to participate**, and silence manifests as an absence of "healthy" evidence rather than as a successful forgery of it.

Simulations and Results

The scheme was evaluated on the **OMNeT++ v5.6.2 simulation platform with the INET framework**, under a configuration chosen to stress its dynamic behavior. The minimum coverage times were measured at the 85%, 90%, and 95% levels; the simulation area was scaled in proportion to the number of devices—starting from a 125×125 m² field for 128 devices and growing to 500×500 m² for 2,048 devices—so that device density remained roughly constant as the swarm grew; the broadcast frequency was 500 ms; devices moved with random mobility at about 10–15 meters per second; and each reported figure is the average of 50 runs.

The Bloom-filter sizing, drawn from the simulation parameters, shows the structure's economy and its tunability against the false-positive rate. Assuming

10% of the devices are compromised and tolerating a **1% false-positive probability**, the number of compromised devices n_c scales with the network (13, 26, 52, 103, 205 for networks of 128, 256, 512, 1024, 2048 devices), and the number of bits required m grows roughly in proportion (125, 250, 499, 988, 1965), with 7 hash functions throughout and the filter needing between 1 and 3 fragments to transmit as the network grows. Relaxing the tolerance to a **5% false-positive probability** shrinks the filter substantially for the same networks: m drops to 70, 139, 309, 611, 1246 bits, with only 4 hash functions and 1 to 2 fragments required. The trade-off is explicit—accepting a higher false-positive rate buys a smaller, cheaper-to-transmit filter.

The run-time results confirm both the scheme's scalability and its advantage over PADS. Plotting run-time against the number of devices for the three coverage targets (85%, 90%, 95%), all curves rise together—low and flat for small networks, climbing more steeply toward roughly 5–6 seconds at 2,000 devices under the 1% false-positive setting, and toward a lower 2.5–3 seconds under the more permissive 5% setting—with the three coverage targets lying close together. Most tellingly, when the Bloom-filter scheme is compared directly against PADS at the 95% coverage level, PADS's run-time rises steeply with network size while the **Bloom-filter scheme's run-time rises far more gently**, so that at around a thousand devices PADS is markedly slower. The Bloom-filter representation thus not only restores flexible membership but also propagates faster than PADS's fixed bitmask, under both the 1% and 5% false-positive regimes.

Bloom-Filter CRA: Advantages and Limitations

The Bloom-filter-based scheme can finally be assessed against the whole lineage that produced it. On the positive side, it retains **all of PADS's advantages**—suitability for dynamic networks, tolerance of device movement during attestation, and a run-time network snapshot for the verifier—while being **faster than PADS**. Decisively, it is **flexible in the joining and leaving of devices**: because a Bloom filter is not bound to a fixed per-device index the way PADS's 2N-bit mask was, a new device can be folded into the filter without resizing a global structure or reconfiguring every node, dissolving the membership rigidity that was PADS's chief flaw. And, like its consensus-based predecessor, it **provides a global view of the network state**.

The limitations are the honest costs of these gains. The scheme relies on **shared symmetric keys**, a simplification adopted because, as in the dynamic setting generally, storing the individual public keys of all devices proved too expensive—so

a single secret key shared by all the nodes is used instead, which concentrates trust in that shared key. The approach is **probabilistic**, inheriting the Bloom filter's false positives, which means **the compromised devices cannot be identified exactly**—a device may be flagged as compromised when it is merely a false positive of the filter. And because the consensus is reached through opaque, accumulating gossip, **it is not known how many nodes actually participated in the consensus**, leaving a residual uncertainty about how thoroughly any particular verdict has been corroborated. These are the characteristic trade-offs of trading exact, heavyweight accounting for lightweight, flexible, probabilistic scalability—the bargain that makes attestation practical for the fluid, open, resource-starved swarms that increasingly populate the Internet of Things.